

Un perfil de integración para el intercambio gobernado de modelos de IA en espacios de datos: la ontología DAIMO

Journal Title
XX(X):1–22
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Edmundo de Elvira Mori Orrillo¹ y Jiayun Liu¹

Abstract

El despliegue interorganizacional de modelos de inteligencia artificial exige un tratamiento semántico coherente de cinco tareas hoy fragmentadas entre vocabularios distintos: publicar el modelo como activo de catálogo, seleccionarlo bajo una política de uso, invocarlo mediante un servicio tipado, trazar cada ejecución y compararlo sobre condiciones de evaluación reproducibles. MLDCAT-AP describe el modelo, DCAT lo publica, ODRL expresa la política, PROV-O registra la procedencia y el *Dataspace Protocol* (DSP) negocia el intercambio contractual; ninguno articula las cinco tareas como cadena única.

Este artículo presenta DAIMO (*Dataspace AI Model Ontology*), un perfil de integración OWL 2 DL que reutiliza los vocabularios citados mediante alineamientos axiomáticos verificados. DAIMO introduce catorce clases nativas (nueve de primer nivel más cinco subclases de rol) para cubrir relaciones puente no expresadas directamente por los vocabularios reutilizados, y se acompaña de una capa SHACL con restricciones estructurales, seis invariantes SHACL-SPARQL entre clases y un banco de pruebas negativas.

DAIMO se desarrolla siguiendo la metodología *Linked Open Terms* (LOT) y se ejercita sobre un demostrador multi-participante que instancia las veintitrés preguntas de competencia formuladas durante la especificación de requisitos. La evaluación reporta consistencia OWL, materialización OWL-RL, conformidad SHACL, pruebas negativas, consultas SPARQL y un benchmark sintético de escalabilidad acotada.

Keywords

gobernanza de modelos de IA; espacios de datos; alineamiento ontológico; validación SHACL; metodología LOT

Introducción

El Reglamento (UE) 2024/1689, conocido como *AI Act*, exige para los sistemas de inteligencia artificial de alto riesgo documentación técnica, registro de ejecuciones, trazabilidad y evidencia de auditoría⁴¹. Paralelamente, los espacios de datos europeos promueven intercambios contractuales entre participantes autónomos mediante catálogos, políticas y protocolos como el *Dataspace Protocol* (DSP)³⁷, implementado en conectores como Eclipse Dataspace Components (EDC)³⁹. En este contexto, un modelo de IA no es sólo un artefacto técnico: es un activo gobernado que debe poder publicarse, descubrirse, invocarse, trazarse y evaluarse a través de fronteras organizativas.

Los vocabularios existentes cubren fragmentos importantes de ese ciclo. MLDCAT-AP 3.0.0³⁵ extiende DCAT para describir modelos de aprendizaje automático como activos de catálogo, con metadatos sobre tareas, ejecuciones, evaluaciones, riesgos y recursos computacionales. DCAT² aporta la capa de catálogo y servicios; ODRL⁴, la representación de ofertas, permisos, prohibiciones y acuerdos; PROV-O³, la procedencia de entidades, actividades y agentes; y DSP, el intercambio contractual en espacios de datos. Las *model cards*⁶, *factsheets*⁷ y *datasheets*¹⁹ complementan estos vocabularios con documentación narrativa.

La brecha identificada en la literatura revisada es relacional. Un modelo descrito con MLDCAT-AP no queda,

por ello solo, conectado al acto de ser ofrecido en un catálogo federado. Una política ODRL puede expresar permisos sin quedar ligada al despliegue que gobierna. Una negociación DSP no explicita necesariamente el contexto de evaluación ni la evidencia de auditoría asociada a una invocación de IA. Un bundle PROV puede agrupar procedencia sin distinguir la cadena que cruza varios contextos administrativos. El problema no es añadir otra descripción del modelo, sino articular catálogo, política, despliegue, ejecución, evidencia y evaluación como una cadena consultable y validable.

Este artículo presenta DAIMO (*Dataspace AI Model Ontology*), un perfil OWL 2 DL de integración para modelos de IA gobernados en espacios de datos. DAIMO sigue una estrategia de reutilización preferente: conserva `it6:MachineLearningModel` como clase de modelo de MLDCAT-AP y utiliza DCAT, ODRL, PROV-O, DSP/EDC, FOAF, SKOS y SPDX allí donde ya proporcionan la semántica necesaria. Sólo introduce clases puente cuando la relación entre vocabularios no queda cubierta por esos

¹Departamento de Inteligencia Artificial, Escuela Técnica Superior de Ingenieros Informáticos, Universidad Politécnica de Madrid, España

Corresponding author:

Edmundo de Elvira Mori Orrillo, Universidad Politécnica de Madrid, Campus de Montegancedo, 28660 Boadilla del Monte, Madrid, España.
Email: edmundo.mori.orrillo@upm.es

recursos. Los requisitos y el demostrador se anclan en la iniciativa de espacio de datos PIONERA, cuyos casos de uso de Movilidad y FLARES proporcionan un contexto de aplicación único y coherente a lo largo del artículo.

Las contribuciones se organizan en tres partes:

1. un perfil de integración *reuse-first* con catorce clases nativas, de las cuales nueve son clases de primer nivel y cinco son subclases de rol, orientadas a cubrir relaciones puente entre catálogo, política, despliegue, ejecución, procedencia y evaluación;
2. una evaluación reproducible del artefacto ontológico mediante consistencia OWL, materialización OWL-RL, verificación de implicaciones, conformidad SHACL, banco negativo de invariantes, escaneo OOPS!, veintitrés consultas de competencia y un benchmark sintético de escalabilidad acotada;
3. un demostrador multi-participante, deliberadamente sintético, que instancia el perfil sobre un escenario de predicción de riesgo y permite evaluar la adecuación representacional sin presentarlo como despliegue productivo ni validación externa por expertos.

La evidencia reportada es, por tanto, evidencia ejecutable sobre el artefacto ontológico y sobre un grafo de demostración. No se interpreta como certificación legal, adopción institucional ni validación de una plataforma federada en producción. El resto del artículo revisa el trabajo relacionado, describe la metodología y los requisitos, presenta la ontología, reporta la evaluación, discute límites y rutas de evolución, y resume la disponibilidad del artefacto.

Trabajo relacionado

DAIMO se sitúa en la intersección de cuatro familias de trabajo que la literatura existente trata habitualmente por separado: documentación narrativa de modelos, ontologías del ciclo de vida de aprendizaje automático, vocabularios de gobernanza y estándares de espacios de datos. Revisamos cada familia por orden de proximidad al problema, identificando qué aporta al puente catálogo-dataspace, dónde lo deja abierto y cómo DAIMO se posiciona respecto a ella. Una quinta subsección revisa las ontologías SWJ recientes cuya metodología de validación hemos adoptado, antes de cerrar la sección con una matriz comparativa de reuso inspirada en RePlanIT⁹.

Descripción de modelos de IA

Documentación narrativa. Las *model cards* de Mitchell et al.⁶, las *factsheets* de Arnold et al.⁷ y las *datasheets for datasets* de Gebru et al.¹⁹ elevaron los estándares de transparencia sobre artefactos de IA. Describen en lenguaje natural el uso previsto del modelo, sus limitaciones, el rendimiento esperado sobre benchmarks conocidos y las condiciones bajo las que ha sido entrenado y evaluado. Son valiosas para la comunicación con lectores humanos y para la rendición de cuentas cualitativa, pero insuficientes como sustrato para descubrimiento automatizado, filtrado por política o vinculación de evidencia en flujos máquina a máquina: un espacio de datos requiere primitivas que los

agentes de software puedan consumir, no documentos que otros agentes deban interpretar.

Ontologías del ciclo de vida ML. Allí donde la documentación narrativa falla en el consumo máquina, las ontologías estructuradas del ciclo de vida ML ocupan su lugar. EXPO²² ofrece una ontología general de experimentos científicos que ha servido de base a trabajos posteriores. ML-Schema²³ formaliza la semántica de experimentos de aprendizaje automático mediante un esquema reutilizable y ligero. MEX²⁴ provee un vocabulario de intercambio para resultados de experimentos, y OntoDM²⁵ junto con DMOP²⁶ articulan el dominio de la minería de datos y el *semantic meta-mining*. Souza et al.³⁰ y Schmidt et al.³¹ extienden la perspectiva hacia la procedencia y la FAIRness del ciclo de vida ML. Estas ontologías proporcionan un fundamento riguroso pero permanecen centradas en el experimento científico y en el proceso de entrenamiento; ninguna aborda la dimensión contractual del intercambio entre participantes autónomos.

Un grupo reciente de trabajos se desplaza desde el experimento hacia la taxonomía y gestión de activos de IA. La *Artificial Intelligence Ontology* (AIO) sistematiza conceptos técnicos y éticos de IA en ramas como sesgo, capas, tareas de aprendizaje automático, funciones matemáticas, modelos, redes, preprocesamiento y estrategias de entrenamiento²⁷. La *Intelligence Task Ontology* (ITO) organiza tareas, benchmarks y métricas de rendimiento para analizar el progreso de capacidades de IA a gran escala²⁸. En la misma línea, Novacek et al. proponen una gestión ontológica de modelos y datasets orientada a relacionar modelos con los datos usados para su entrenamiento y validación²⁹. Estos enfoques refuerzan la necesidad de describir la estructura técnica del modelo, sus datos y sus métricas; DAIMO aborda una pregunta distinta y complementaria: cómo convertir esas descripciones en un activo ofertable, autorizable, invocable, auditable y comparable dentro de un espacio de datos federado.

El trabajo más próximo al dominio de DAIMO, y el más reciente, es MLDCAT-AP 3.0.0³⁵, publicado por SEMIC en 2025. MLDCAT-AP extiende DCAT para describir modelos de IA como subclases de `dcat:Dataset`, con metadatos ricos sobre política, benchmark, evaluación, ejecución, riesgo e infraestructura computacional. DAIMO lo reutiliza directamente: la clase `it6:MachineLearningModel` es la clase del modelo en DAIMO, y las clases `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation` e `it6:ComputerInfrastructure` capturan respectivamente la tarea objetivo, la ejecución concreta, el *pipeline* asociado, la medición resultante y el entorno de cómputo. MLDCAT-AP es fuerte en el lado del modelo pero deliberadamente agnóstico respecto al espacio de datos en el que el modelo circula: no modela el acto de ofrecer un modelo a través de fronteras organizativas, ni las nociones de despliegue gobernado, autorización de ejecución anclada a un acuerdo o procedencia agregada entre contextos administrativos distintos. MLDCAT-AP cubre el modelo; DAIMO cubre el puente entre el modelo y el dataspace.

La Tabla 1 explicita la relación clase a clase entre DAIMO y MLDCAT-AP. El patrón es intencional: DAIMO adopta las clases de MLDCAT-AP para el plano del activo de IA, añade restricciones de perfil cuando el

intercambio gobernado exige completitud operativa, e introduce extensiones sólo para las relaciones puente que MLDCAT-AP no cubre. La versión evaluada del perfil se mantiene sobre MLDCAT-AP 3.0.0 porque los alineamientos, ejemplos y pruebas SHACL fueron contruidos contra esa especificación. MLDCAT-AP 3.1.0, publicada como *Candidate Recommendation* en 2026³⁶, queda como riesgo de evolución que deberá contrastarse en futuras versiones del alineamiento.

El bloque superior de la Tabla 1 confirma que DAIMO no redefine ni bifurca la descripción del modelo: todos los constructos de MLDCAT-AP se adoptan, y los casos perfilados sólo añaden obligaciones de uso en lugar de nueva semántica. El bloque inferior aísla los constructos puente que MLDCAT-AP no proporciona, que son exactamente las relaciones de gobernanza que motivan el perfil. Leídos en conjunto, ambos bloques muestran que la contribución es relacional y no un vocabulario de modelo competidor.

Gobernanza de recursos y espacios de datos

Vocabularios W3C de gobernanza. Las ontologías del ciclo de vida describen qué es un modelo y cómo se produjo, pero no gobiernan cómo se publica, se licencia o se atribuye. Tres vocabularios W3C proporcionan ese sustrato de gobernanza. DCAT 2² proporciona las primitivas de catálogo sobre las que se apoyan tanto MLDCAT-AP como DAIMO. ODRL 2.2⁴ ofrece un modelo consolidado para expresar permisos, prohibiciones, obligaciones, ofertas (`odrl:Offer`) y acuerdos (`odrl:Agreement`) vinculantes; DAIMO lo usa intensivamente tanto para la política adherida a las ofertas como para los acuerdos que autorizan ejecuciones concretas. PROV-O³ aporta la ontología de procedencia con actividades, entidades, agentes, bundles y roles, y es la capa natural sobre la que codificar la trazabilidad de ejecuciones individuales y su agregación entre participantes. Pandit y Esteves¹⁵ muestran cómo ODRL se combina con DPV para escenarios de salud, un patrón que DAIMO transpone al dominio IA. Estos vocabularios son maduros y ampliamente adoptados, pero por construcción son agnósticos al dominio: ninguno es específico de inteligencia artificial ni de espacios de datos.

Estándares de espacios de datos. Mientras que los vocabularios anteriores describen artefactos, los estándares de espacios de datos describen el intercambio en sí. La perspectiva desplaza el foco desde los artefactos aislados hacia los intercambios gobernados entre actores autónomos. Franklin et al.¹ formularon las *dataspace*s como respuesta a la heterogeneidad persistente de fuentes. Otto et al.¹⁷ y Cappiello et al.¹⁸ convierten la soberanía del dato, los contratos y la interoperabilidad controlada en supuestos explícitos del diseño. El *Dataspace Protocol*³⁷ es la especificación W3C-CG que define los mensajes de catálogo, la negociación de contratos y los procesos de transferencia; Eclipse Dataspace Components³⁹ es su implementación de referencia.

Conviene distinguir entre *framework* y *vocabulario*: EDC es un framework de ejecución construido en Java, no una ontología. La semántica que EDC implementa proviene de DSP (protocolo), ODRL (políticas) y DCAT (catálogo); sólo un puñado de conceptos son verdaderas

extensiones específicas de EDC, y de ellos DAIMO referencia explícitamente `edc:ParticipantContext` por su utilidad para anclar la procedencia agregada a un contexto administrativo. Esta aclaración es importante porque en la literatura de espacios de datos las dos capas se confunden con frecuencia. La versión evaluada de DAIMO conserva los términos DSP utilizados durante el desarrollo; la evolución de DSP 2025-1³⁸ constituye un punto de seguimiento para futuras versiones del alineamiento informativo. Trabajos sobre ecosistemas federados como ConSolid⁸ muestran además que la Web Semántica evoluciona hacia escenarios multi-actor donde gobernanza e interoperabilidad deben diseñarse conjuntamente.

Ingeniería ontológica ejecutable en SWJ

Más allá del sustrato de dominio que las cuatro familias anteriores proporcionan, un quinto hilo de trabajo reciente informa cómo debe evaluarse DAIMO. El *Semantic Web Journal* ha publicado recientemente un conjunto de ontologías que comparten un compromiso metodológico explícito con la validación ejecutable y la reproducibilidad. Kurteva et al. presentan RePlanIT⁹, una ontología para pasaportes digitales de producto de equipos ICT, con SHACL y evaluación de usabilidad con expertos de industria. Palma et al. operacionalizan información de suelos a escala global en GloSIS¹⁰ mediante un perfil modular alineado con SOSA/SSN e ISO 28258. Woods et al. conectan ingeniería ontológica con procesamiento del lenguaje natural para actividades de mantenimiento industrial¹¹. Bareedu et al. derivan reglas SHACL desde estándares industriales OPC UA¹³. Ferranti et al. formalizan restricciones de propiedad de Wikidata mediante SHACL y SPARQL¹⁴. Robaldo y Batsakis¹⁶ analizan la interacción entre validación SHACL e inferencia sobre la Ontología del Tiempo. Penteado et al.¹² estudian metodologías de publicación de linked open government data. Estos trabajos establecen colectivamente la expectativa —que DAIMO adopta como vinculante— de que una ontología con pretensiones operacionales presente evidencia reproducible de su corrección, no solamente una descripción axiomática.

La comparación se condensa ahora en una matriz de reuso. La tabla permite leer, en una sola vista, qué familias cubren publicación, política, invocación, trazabilidad y evaluación, y por qué DAIMO se formula como perfil de integración antes que como sustituto de esos vocabularios.

La lectura horizontal de la Tabla 2 indica que ningún artefacto individual cubre simultáneamente las cinco dimensiones; la lectura vertical confirma que cada columna está cubierta nativamente por al menos un vocabulario reutilizable. DAIMO se diseña, por tanto, como un perfil de integración que ata esas cinco capacidades a través de alineamientos axiomáticos verificados, sin duplicar lo que los vocabularios maduros ya resuelven.

Metodología y requisitos

DAIMO adopta la metodología *Linked Open Terms* (LOT)³² por dos razones. Primera, su orientación industrial: LOT prescribe un flujo ligero con entregables en cada una de sus cuatro fases (especificación de requisitos, implementación, publicación y mantenimiento). Segunda,

Tabla 1. Relación entre DAIMO y MLDCAT-AP 3.0.0.

Constructo	Tratamiento DAIMO	Notas
it6:MachineLearningModel	Adoptada + perfilada	Descripción del activo de IA; DAIMO exige política, título e identificador para intercambio gobernado.
it6:Task	Adoptada	Tarea objetivo del modelo.
it6:Run	Adoptada + perfilada	Registro de ejecución; DAIMO exige <i>flow</i> , algoritmo, agente, marca temporal y enlace de autorización.
it6:Flow	Adoptada	Pipeline asociado a una ejecución.
it6:Evaluation	Adoptada	Resultado de medición; puede formar parte de la procedencia cruzada.
it6:EvaluationMeasure	Adoptada	Tipo y valor de métrica.
it6:Benchmark	Adoptada	Referencia de benchmark para condiciones de comparabilidad.
it6:ComputerInfrastructure	Adoptada	Entorno de cómputo donde se ejecuta un despliegue.
it6:Hardware, it6:Library	Adoptadas	Componentes del entorno de cómputo.
it6:HarmRisk	Adoptada	Anotación de riesgo sobre el modelo.
it6:Modality	Adoptada	Modalidad de entrada.
it6:trainedOn, it6:testedOn	Adoptadas	Enlaces a datasets de entrenamiento y prueba.
Constructos puente ausentes en MLDCAT-AP		
daimo:AIAssetOffering	Extensión puente	Oferta de catálogo gobernada con política ODRL asociada.
daimo:ExecutionAuthorization	Extensión puente	Autorización de ejecución basada en un acuerdo.
daimo:ModelDeployment	Extensión puente	Relación de despliegue entre modelo, infraestructura y servicio invocable.
daimo:IOContract	Extensión puente	Formatos de entrada/salida y condiciones de autenticación.
daimo:DerivedArtifact	Extensión puente	Salida gobernable de una ejecución autorizada.
daimo:AuditEvidence	Extensión puente	Evidencia de integridad y auditoría orientada a cumplimiento.
daimo:CrossParticipantProvenanceRecord	Extensión puente	Agregación de procedencia entre contextos de participante.
daimo:SharedEvaluationContext	Extensión puente	Condiciones reificadas de comparabilidad de evaluaciones.
daimo:ParticipantRole	Extensión puente	Rol de participante en el intercambio dentro del dataspace.

Tabla 2. Matriz de reuso frente a los atributos operacionales requeridos. ● = cubierto nativamente; ○ = cubierto parcialmente; — = no cubierto.

Familia / artefacto	Publicación	Política	Invocación	Trazabilidad	Evaluación
Model cards, factsheets, datasheets ^{6,7,19}	○	—	—	—	○
ML-Schema, MEX, OntoDM, DMOP ^{23–26}	—	—	—	○	●
AIO, ITO, gestión modelo-dataset ^{27–29}	—	—	—	○	●
MLDCAT-AP 3.0.0 ³⁵	●	○	—	●	●
DCAT 2 ²	●	—	○	—	—
ODRL 2.2 ⁴	—	●	—	—	—
PROV-O ³	—	—	—	●	○
DSP / EDC ^{37,39}	○	○	●	—	—
DAIMO (este trabajo)	●	●	●	●	●

LOT trata la reutilización como fase de primera clase, no como subproducto, lo que encaja con la regla reuse-first que el problema impone. Las subsecciones siguientes describen el escenario operativo que guió la elicitación de requisitos (§), las preguntas de competencia y los requisitos no funcionales resultantes (§), y las tres decisiones centrales de diseño que dieron forma a la ontología (§).

Escenario ejecutivo

La especificación de requisitos se ancla en el espacio de datos PIONERA, una iniciativa española sobre la mejora de la interoperabilidad en espacios de datos mediante inteligencia artificial, que posteriormente actúa como caso de estudio (§). En PIONERA, un conector de Empresa actúa como proveedor de modelos y un conector de Ayuntamiento

actúa como consumidor municipal, coordinados por un operador de plataforma que gestiona el catálogo federado y la negociación entre conectores. El catálogo expone dos familias de modelos que motivan los requisitos: los modelos de Movilidad, que predicen tiempo de viaje y retraso del transporte público a partir de características de tramo tipo GTFS, y los modelos FLARES, que extraen eventos y clasifican fiabilidad sobre texto en español. El consumidor municipal necesita invocar un modelo de Movilidad de predicción de retraso para apoyar decisiones operativas, pero no puede hacerlo sin antes descubrirlo bajo una política compatible con su uso público, verificar que satisface un umbral mínimo de calidad sobre un contexto de evaluación común, negociar un acuerdo que autorice sus ejecuciones concretas e invocarlo a través de un endpoint autenticado. Un servicio de benchmarking compara el modelo con una línea base sobre el mismo contexto de evaluación, y un actor de gobernanza audita las ejecuciones y archiva la evidencia correspondiente. Los nombres, modelos, endpoints, métricas y fechas del escenario son sintéticos por diseño: el caso no se presenta como piloto desplegado, sino como demostrador reproducible de adecuación representacional.

Este escenario sintetiza las cinco tareas operativas que una ontología del puente catálogo-dataspace debe soportar simultáneamente (publicación, descubrimiento, invocación, trazabilidad y evaluación) e involucra a cinco tipos de actor claramente diferenciados cuyas necesidades son complementarias pero no coinciden. Es verosímil: no depende de tecnología hipotética y se puede instanciar con los vocabularios reutilizados más las extensiones que DAIMO introduce.

Especificación de requisitos

Los requisitos se elicitaron desde tres fuentes complementarias: el escenario anterior, la especificación y notas de implementación de MLDCAT-AP 3.0.0 y el análisis de términos seleccionados de las extensiones Eclipse EDC empleadas en plataformas de espacios de datos. A partir de ellas se formularon veintitrés preguntas de competencia, organizadas en cinco categorías por actor y etapa del ciclo de vida. La categoría R (registro y publicación, cinco preguntas) cubre las necesidades del proveedor; la categoría D (descubrimiento y selección, cuatro preguntas), las del consumidor; la categoría E (ejecución y auditabilidad, cinco preguntas), las del operador y gobernanza; la categoría V (evaluación y reproducibilidad, cinco preguntas), las del evaluador; y la categoría G (puente de gobernanza, cuatro preguntas), las preguntas multi-actor que requieren semántica nativa de DAIMO. El conjunto completo de las veintitrés preguntas en lenguaje natural aparece en el Apéndice .

La primera síntesis vincula actores y familias de preguntas antes de introducir cualquier trazabilidad técnica. La Tabla 3 asocia cada rol de tipo PIONERA —proveedor, consumidor, operador de plataforma, evaluador y la perspectiva de gobernanza multi-actor— con la familia de preguntas de competencia que origina y con la necesidad concreta que hay detrás. Presentar primero a los actores fija quién formula cada requisito y evita que las preguntas de competencia se lean como una lista abstracta desvinculada del escenario. Además deja explícito que la categoría de gobernanza

es genuinamente transversal, porque ningún participante aislado puede responderla por sí solo.

El segundo paso es trazar cada familia de requisitos hacia el compromiso de modelado que la satisface y hacia la evidencia que después lo demuestra. La Tabla 4 es, por tanto, la bisagra entre la fase de especificación LOT y la fase de evaluación: para cada requisito operativo nombra el constructo de la ontología que porta la semántica y el artefacto de validación —consulta de competencia, *shape* SHACL o prueba negativa— que lo comprueba. Leer la tabla de arriba abajo justifica por qué existe cada bloque principal de la ontología, en lugar de presentar las clases como preferencias de diseño. También anticipa la sección de evaluación, de modo que el lector ya puede ver qué requisito pretende apoyar cada resultado posterior.

El flujo operacional de ejecución federada se interpreta en DAIMO como una cadena semántica auditable, no como una arquitectura de runtime. El descubrimiento se representa mediante ofertas gobernadas y CQs D1–D4; la negociación, mediante `odrl:Agreement` y `ExecutionAuthorization`; el despliegue, mediante `ModelDeployment`, `dcat:DataService` e `IOContract`; la inferencia, mediante `it6:Run` y `DerivedArtifact`; y el cierre operativo o *clearing*, mediante `AuditEvidence` y `CrossParticipantProvenanceRecord`. Esta lectura preserva la premisa de soberanía: DAIMO puede describir ejecuciones en el nodo del proveedor de datos, en un nodo neutral o en una infraestructura acordada, pero no prescribe si el modelo viaja al dato, el dato al modelo, o si la ejecución ocurre dentro de un mecanismo particular de aislamiento.

La enumeración de términos se trató como un artefacto intermedio entre las preguntas de competencia y la TBox. La Tabla 5 no añade semántica nueva: normaliza el vocabulario operativo que aparece en el escenario, decide si cada término se reutiliza o se especializa, y evita convertir temas generales en clases nativas sin necesidad. Las definiciones formales completas se mantienen en la ontología mediante `skos:definition`, `skos:example`, dominios, rangos y *shapes*.

El conjunto de CQs combina consultas directas, rutas de subclase o subpropiedad, filtros, agregaciones y negación como fallo. Esta variedad es deliberada: un perfil que se apoyara sólo en consultas directas no ejercitaría la parte del artefacto que hace valiosa la semántica OWL, y un perfil que exigiera razonamiento materializado para toda pregunta sería impracticable en operación sobre endpoints reales. Los requisitos no funcionales complementan los funcionales y fijan: perfil OWL 2 DL (para mantener decidibilidad y compatibilidad con HermiT), licencia CC-BY 4.0 para la ontología y Apache 2.0 para el código de validación, identificadores persistentes bajo <https://w3id.org/pionera/daimo>, serializaciones RDF estándar y validación SHACL accesible.

Decisiones centrales de diseño

Tres decisiones estructurales se tomaron en la fase de conceptualización y condicionan el resto del diseño.

Tabla 3. Actores del escenario y categorías de preguntas de competencia.

Actor	Categoría	Necesidad	CQs
Proveedor de modelos	R	Publicar un modelo como activo gobernado con identidad, licencia, política y contrato de invocación	CQ-R1–CQ-R5
Consumidor de modelos	D	Descubrir modelos por tarea, dominio, política y umbral de calidad	CQ-D1–CQ-D4
Operador de plataforma	E	Invocar servicios, trazar ejecuciones y reconstruir evidencia operativa	CQ-E1–CQ-E5
Evaluador	V	Comparar resultados en un contexto compartido e inspeccionar reproducibilidad	CQ-V1–CQ-V5
Gobernanza (multi-actor)	G	Puentear oferta, despliegue, autorización y procedencia entre participantes	CQ-G1–CQ-G4

Tabla 4. Trazabilidad entre requisitos operacionales, compromisos de modelado y evidencia.

Familia de requisito	Pregunta que el perfil debe responder	Compromiso DAIMO principal	Evidencia
Publicación	¿Este modelo es un activo gobernado en un contexto de catálogo concreto?	Registro de oferta que vincula modelo, proveedor, contexto de catálogo y política aplicable.	CQs R1–R5, G1
Descubrimiento	¿Puede un consumidor filtrar modelos por tarea, dominio, política y umbral de evaluación?	Semántica de modelo, tarea y evaluación de MLDCAT-AP más enlaces a política y contexto de evaluación.	CQs D1–D4
Invocación	¿Qué servicio, método de autenticación y contrato E/S aplican a un modelo seleccionado?	Despliegue de modelo conectado a servicios DCAT y contratos E/S, permitiendo varios endpoints por modelo alojado.	CQs E1, G2
Autorización	¿Qué acuerdo autorizó una ejecución concreta por un participante concreto?	Acuerdo negociado enlazado al agente beneficiario y a la ejecución que cubre.	CQs E2, G3; INV-2, INV-4
Auditabilidad	¿Puede un actor de gobernanza reconstruir artefactos, evidencias y procedencia entre contextos?	Artefacto derivado, evidencia de auditoría y bundle de procedencia entre participantes.	CQs E4, E5, G4; INV-1
Evaluación	¿Son comparables las mediciones bajo la misma tarea, versión de dataset, protocolo y semilla?	Contexto compartido de evaluación que fija tarea, dataset, versión, protocolo y semilla.	CQs V1–V5

Decisión 1: reutilización preferente. DAIMO reutiliza directamente las clases de MLDCAT-AP para todo aquello que MLDCAT-AP ya modela con precisión. `it6:MachineLearningModel` es la clase del modelo; `it6:ComputerInfrastructure` describe el entorno de ejecución; `it6:Run`, `it6:Flow` e `it6:Evaluation` capturan respectivamente la ejecución concreta, el *pipeline* asociado y la medición resultante; `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService` publican el activo; `odrl:Offer` y `odrl:Agreement` expresan la política y el acuerdo. Introducir equivalentes DAIMO-nativos de cualquiera de estas clases violaría el principio de compromiso ontológico mínimo de Gruber y fracturaría el ecosistema DCAT-AP sin ganancia; una clase DAIMO-nativa sólo se justifica cuando la unión de los vocabularios reutilizados deja un hueco concreto que impide responder a al menos una pregunta de competencia.

Esta decisión ordena el modelo conceptual sin convertir cada tema en una clase nueva. El modelo de IA se mantiene como `it6:MachineLearningModel`; los recursos de datos como `dcat:Dataset` y `dcat:Distribution`; la infraestructura como `it6:ComputerInfrastructure`; las métricas como `it6:Evaluation` y `it6:EvaluationMeasure`;

y la gobernanza como combinación de políticas ODRL, autorizaciones DAIMO, roles de participante y evidencia auditable. El linaje no se reifica como una clase monolítica, sino como una cadena consultable que conecta datos, ejecución, despliegue, artefactos derivados y bundles PROV. Por la misma razón, candidatos habituales como `AIModel`, `DataSet`, `Algorithm`, `ModelVersion`, `hasOwner`, `hasLicense`, `frameworks` o hiperparámetros se reutilizan desde MLDCAT-AP, DCAT, ODRL, PROV-O o ML-Schema, o quedan como metadatos de perfiles externos si ninguna pregunta de competencia del núcleo los exige. Así, DAIMO permanece desacoplada de frameworks concretos como PyTorch o TensorFlow y evita importar modelos IDS o Gaia-X como dependencias del núcleo; esos ecosistemas quedan como contextos de adopción interoperables mediante DCAT, ODRL, PROV-O, DSP y los alineamientos del perfil.

Decisión 2: separación en tres capas complementarias. La arquitectura distingue tres capas que DAIMO conecta pero no sustituye. La capa A es el plano del espacio de datos: DSP con las extensiones EDC puntuales que DAIMO referencia. La capa B es el plano de gobernanza de plataforma: una plataforma de espacio de datos con sus mecanismos de registro federado y políticas operativas. La

Tabla 5. Términos nucleares del dominio y realización ontológica.

Término del dominio	Sentido operacional	Realización ontológica
Modelo de IA	Artefacto entrenado o empaquetado que puede publicarse, evaluarse e invocarse.	Reutilizado mediante la clase de modelo de MLDCAT-AP; DAIMO no duplica la clase del modelo.
Oferta gobernada	Acto de registrar un modelo como activo disponible bajo política en un catálogo federado.	AIAssetOffering enlaza modelo, agente y odrl:Offer; se alinea con dcat:CatalogRecord.
Participante	Agente jurídico u organizativo que actúa dentro del espacio de datos.	foaf:Agent para el agente; ParticipantRole para el rol contextual.
Rol de participante	Papel asumido respecto al activo: proveedor, consumidor, operador, evaluador o gobernanza.	Cinco subclases no disjuntas de ParticipantRole, porque un agente puede asumir varios roles.
Contexto de participante	Ámbito administrativo donde se publica, negocia, opera o audita el intercambio.	Reutilizado como edc:ParticipantContext; DAIMO lo usa para acotar roles y procedencia.
Política y acuerdo	Condiciones de uso de la oferta y resultado de una negociación de acceso.	Reutilización de odrl:Offer y odrl:Agreement; DAIMO especializa sólo la autorización de ejecución.
Despliegue de modelo	Instancia alojada que sirve un modelo mediante uno o más servicios.	ModelDeployment conecta modelo, infraestructura, servicio DCAT y contrato E/S.
Servicio o endpoint	Interfaz accesible para invocar un despliegue.	Reutilizado como dcat:DataService; DAIMO añade el enlace desde el despliegue.
Contrato E/S	Formato de entrada, formato de salida, autenticación y esquemas opcionales.	IOContract con inputFormat, outputFormat, authMethod y esquemas opcionales.
Ejecución autorizada	Invocación concreta cubierta por un acuerdo válido.	it6:Run para la ejecución y ExecutionAuthorization para el acuerdo que la autoriza.
Artefacto derivado	Resultado gobernable producido por una ejecución.	DerivedArtifact, enlazado a la ejecución y a la autorización bajo la que se produjo.
Evidencia de auditoría	Registro verificable que respalda una actividad ejecutada.	AuditEvidence con actividad, firmante, marca temporal y checksum SPDIX.
Procedencia cruzada	Narrativa de procedencia que cruza al menos dos contextos de participante.	Clase DAIMO de procedencia cruzada como bundle PROV con contextos explícitos.
Contexto de evaluación	Condiciones que hacen comparables las métricas de dos modelos.	SharedEvaluationContext fija tarea, dataset, versión, protocolo, semilla y flow.
Evaluación y métrica	Medición de desempeño usada para comparar modelos.	Reutilización de it6:Evaluation; DAIMO enlaza la evaluación al contexto compartido.

capa C es el plano del activo de IA/ML: MLDCAT-AP y los vocabularios que lo soportan. DAIMO es el perfil de integración que conecta A, B y C; no es una cuarta capa paralela que replique funcionalidad, sino un puente que hace consultable la articulación entre las tres.

Decisión 3: una clase nativa por hueco, no por tema. Dada la reutilización preferente, cada clase DAIMO-nativa debe justificar su existencia por un hueco concreto en la unión de los vocabularios reutilizados y por al menos una pregunta de competencia que no puede responderse sin ella. El criterio se aplica con rigor y produce nueve clases de primer nivel (AIAssetOffering, ParticipantRole, ModelDeployment, IOContract, ExecutionAuthorization, DerivedArtifact, CrossParticipantProvenanceRecord, AuditEvidence y SharedEvaluationContext) más cinco subclases de rol. Cualquier clase que no cumpla simultáneamente ambas condiciones queda descartada.

La ontología DAIMO

Arquitectura modular y métricas

DAIMO modela el ciclo de vida operacional de un activo de modelo de IA dentro de un espacio de datos gobernado: publicación como registro de catálogo, descubrimiento sensible a política y a calidad, invocación controlada mediante servicios tipados, trazabilidad de ejecuciones individuales y evaluación contextualizada y reproducible. El alcance se restringe deliberadamente a esta cadena operacional. Las responsabilidades ortogonales, como la modelización detallada del *pipeline* de entrenamiento, la semántica fina de políticas de privacidad o la teoría formal de la derivación PROV, ya están cubiertas por vocabularios consolidados cuya duplicación fragmentaría el ecosistema. DAIMO aporta, en cambio, los constructos necesarios para

que esos vocabularios se articulen coherentemente en el caso concreto de un modelo de IA que circula entre participantes autónomos.

La ontología se estructura en tres módulos complementarios, distribuidos en archivos Turtle separados para que un consumidor pueda cargar únicamente los que necesite. El **módulo núcleo** (`daimo-core.ttl`) contiene las catorce clases y las veintinueve propiedades de objeto más ocho propiedades de datatype DAIMO-nativas, los axiomas globales de disyunción y las caracterizaciones sobre propiedades (funcionales, asimétricas, inversas) que capturan las invariantes de tipo a nivel TBox. El **módulo de alineamientos** (`alignment.ttl`) encapsula ocho declaraciones `rdfs:subClassOf` externas y cinco declaraciones `rdfs:subPropertyOf` externas que conectan DAIMO con los vocabularios reutilizados, más las declaraciones mínimas de los términos externos que DAIMO referencia, cada una con un `rdfs:seeAlso` a la sección correspondiente de la especificación fuente; en conjunto, el paquete contiene seis subpropiedades DAIMO hacia vocabularios externos porque `hasOffering` $\sqsubseteq$ `foaf:isPrimaryTopicOf` se declara en el núcleo como inversa de `offersModel`. El **módulo SHACL** (`daimo-shapes.ttl`) aporta las dieciocho `sh:NodeShape` del perfil: nueve estructurales, tres de conformidad sobre clases reutilizadas y seis invariantes entre clases mediante `sh:SPARQLConstraint`. El módulo se declara como `owl:Ontology` independiente con su propia `owl:versionIRI`, de modo que la capa de validación se libera y referencia como un artefacto con ciclo de vida propio.

La separación en tres módulos no es accidental. El núcleo puede ser consumido por una aplicación que sólo necesite las primitivas DAIMO; el módulo de alineamientos se carga cuando se quiere que un razonador infiera las relaciones entre DAIMO y los vocabularios reutilizados; el módulo SHACL

se carga cuando se quiere validar un grafo concreto contra el perfil.

El mismo diseño modular se resume como perfil de implementación para distinguir decisiones ontológicas de condiciones operativas. La tabla conecta lenguaje, IRIs, validación y acceso con el alcance declarado, y evita interpretar compatibilidad técnica como evidencia de despliegue productivo.

Antes de pasar a las métricas, la Figura 1 resume la organización del paquete DAIMO. Debe leerse de izquierda a derecha: núcleo, alineamientos y SHACL son módulos distribuibles del artefacto, mientras que los vocabularios de la derecha son recursos reutilizados. DSP aparece como protocolo de intercambio y EDC como implementación referenciada por el escenario, no como capas funcionales nuevas.

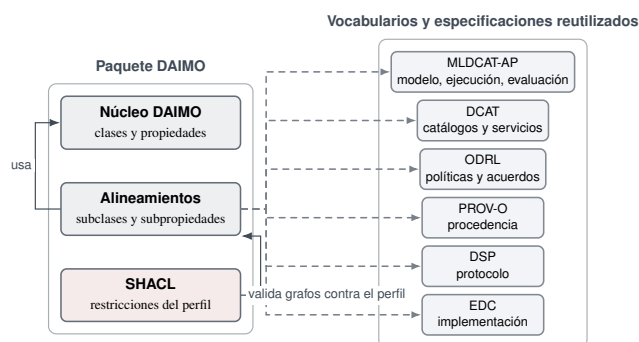


Figura 1. Organización modular del paquete DAIMO y recursos reutilizados.

Métricas del artefacto. Antes de recorrer las clases individuales que pueblan los tres módulos de §, reportamos el tamaño y la expresividad del artefacto globalmente. La Tabla 7 extrae las métricas directamente de los archivos Turtle y de los reportes producidos por la suite de validación (§). Los números permiten a un lector interesado estimar el peso del artefacto antes de consumirlo y compararlo con ontologías publicadas previamente en la misma familia.

Para hacer auditables esos conteos, se conserva un desglose de estilo OntoMetrics separado de la síntesis anterior. La distinción es útil porque evita inflar las cifras con declaraciones externas cargadas sólo para alineamiento y deja claro qué se cuenta como término DAIMO-nativo.

El desglose de la Tabla 8 muestra que las cifras reportadas cuentan sólo sujetos nativos de DAIMO, de modo que las declaraciones de alineamiento que enlazan el perfil con vocabularios externos no inflan los totales. La alta proporción de propiedades funcionales, dominios y rangos refleja un diseño deliberadamente restringido en el que la mayoría de relaciones están tipadas y son de valor único. Los dos ceros de `owl:equivalentClass` y de `owl:disjointWith` binario son esperables, porque la disjunción se expresa mediante un único axioma nombrado `owl:AllDisjointClasses`.

Clases nativas y alineamientos

Las nueve clases nativas de primer nivel, más las cinco subclases de `ParticipantRole`, constituyen el conjunto mínimo que las tres decisiones centrales de diseño

producen. La Tabla 9 actúa como inventario exhaustivo: clase, alineamiento, papel y preguntas de competencia asociadas. Las figuras siguientes no duplican ese inventario; ofrecen una lectura conceptual del conjunto y una lectura axiomática de los alineamientos principales.

La Figura 2 ofrece una lectura a nivel de clases del perfil como diagrama ontológico: cada caja muestra el nombre de una clase y, cuando procede, sus atributos principales, y las flechas etiquetadas son las propiedades de objeto que las conectan. Las clases puente aparecen en azul, las clases externas reutilizadas en gris, las clases de evidencia en ámbar y el contexto de evaluación en verde, de modo que se ve de un vistazo qué conceptos introduce DAIMO y cuáles toma de vocabularios consolidados. El diagrama se restringe deliberadamente a las clases y relaciones más importantes de la cadena gobernada —publicación, despliegue, invocación, autorización, ejecución, evidencia y evaluación— en lugar de reproducir todos los términos. Debe leerse como el mapa conceptual que el resto de la sección desarrolla grupo a grupo.

La Figura 3 complementa la vista global con el nivel axiomático, de modo que ambas figuras se lean juntas: la anterior muestra qué conecta DAIMO y esta muestra cómo se declara esa conexión en OWL. El panel superior separa las clases puente DAIMO de las clases reutilizadas a las que se adhieren mediante `rdfs:subClassOf`, haciendo explícita la superficie de alineamiento. El panel inferior retiene sólo las relaciones operativas principales que articulan el perfil, dejando el listado exhaustivo de términos a la Tabla 9. Separar alineamientos y relaciones en dos paneles mantiene cada lectura despejada.

Los párrafos siguientes explican las decisiones de modelado por función. No repiten la trazabilidad completa de la Tabla 9; se concentran en por qué cada agrupación existe y qué frontera semántica preserva.

Publicación y gobernanza.

`AIAssetOffering` representa el acto de registrar un modelo como oferta gobernada en un catálogo de espacio de datos. La clase no reemplaza al modelo: el modelo sigue siendo `it6:MachineLearningModel`, reutilizado de MLDCAT-AP, y la oferta sólo contextualiza su publicación con proveedor, política ODRL y metadatos de catálogo. `ParticipantRole` captura el papel contextual que un agente asume respecto del activo; sus cinco subclases (`ModelProvider`, `ModelConsumer`, `PlatformOperator`, `Evaluator` y `GovernanceActor`) no son disjuntas, porque un mismo agente puede desempeñar varios papeles en contextos distintos. El perfil no pretende ser un catálogo de catálogos: sólo modela la oferta gobernada de modelos de IA dentro del intercambio descrito.

Despliegue e invocación.

`ModelDeployment` e `IOContract` conectan el modelo reutilizado con el punto de invocación. El despliegue identifica la instancia alojada o en ejecución del modelo y puede exponerse mediante uno o varios `dcat:DataService`; el contrato E/S describe formatos, autenticación y referencias opcionales a esquemas. DAIMO registra dónde y bajo qué condiciones sintácticas se puede invocar un modelo, pero no implementa el runtime, el motor de enforcement, la

Tabla 6. Perfil técnico de implementación de DAIMO.

Pilar	Implementación en DAIMO	Decisión de alcance
Lenguaje, serialización e IRIs	Núcleo OWL 2 DL en Turtle; serializaciones RDF/XML, JSON-LD y N-Triples preparadas para publicación; namespace canónico https://w3id.org/pionera/daimo .	La reproducibilidad local no depende de un endpoint persistente activo.
Vistas conceptuales de implementación	Metadatos de modelo: MLDCAT-AP; datos y catálogo: DCAT; ejecución e infraestructura: MLDCAT-AP más ModelDeployment/IOContract; gobernanza y confianza: ODRL, autorizaciones, evaluación contextualizada y evidencia.	Estas son vistas funcionales, no módulos ontológicos nuevos como ModelMetadata o GovernanceTrust.
Relaciones críticas	Se reutiliza <code>it6:trainedOn</code> ; los formatos y esquemas se expresan en <code>IOContract</code> ; el despliegue se conecta al endpoint mediante <code>exposedAs</code> y <code>dcat:DataService</code> ; la conformidad normativa puede expresarse con <code>dct:conformsTo</code> o perfiles ODRL/SHACL específicos.	No se añaden propiedades duplicadas como <code>trainedOn</code> , <code>deployedIn</code> o <code>compliesWith</code> si ya existe una ruta estándar o perfilable.
Validación y razonamiento	SHACL estructural, SHACL sobre clases reutilizadas, seis invariantes SHACL-SPARQL, HermiT, OWL-RL, OOPS!, pruebas negativas y 23 consultas SPARQL.	No se exige una métrica universal como <i>accuracy</i> ; las métricas obligatorias dependen del contexto de evaluación y de la CQ ejercitada.
Almacenamiento y acceso	El paquete RDF puede cargarse en triplestores estándar y exponerse por SPARQL; el conjunto de consultas de competencia demuestra el patrón de acceso.	El artículo no afirma un endpoint productivo GraphDB/Fuseki; sólo compatibilidad técnica con ese tipo de despliegue.

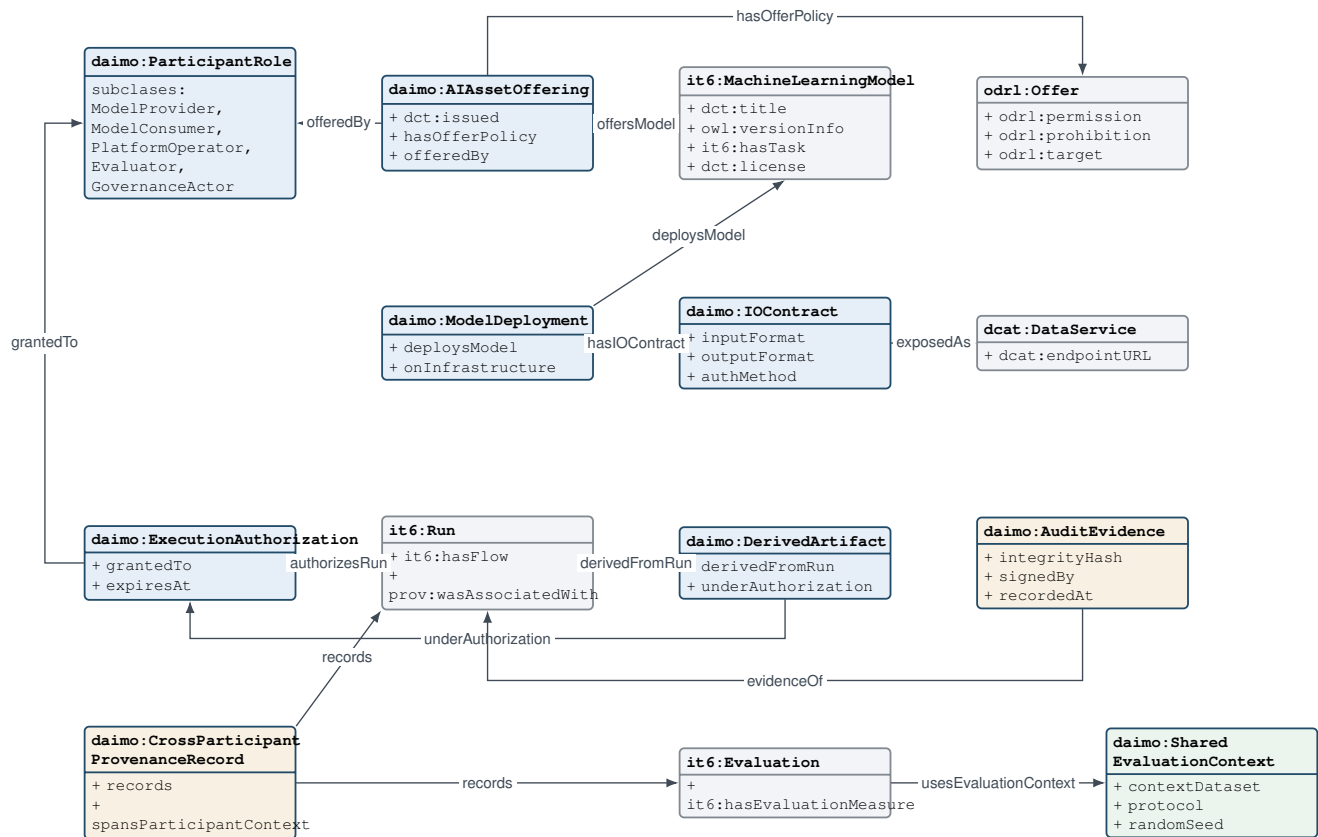


Figura 2. Vista ontológica global de DAIMO: clases puente propias (azul), clases externas reutilizadas (gris), clases de evidencia (ámbar) y el contexto de evaluación compartido (verde), con sus atributos principales y propiedades de objeto.

orquestración de contenedores ni mecanismos TEE. Tampoco resuelve por sí mismo la compatibilidad semántica fina entre esquemas de entrada y salida; esa verificación pertenece a perfiles o *shapes* específicos del espacio de datos.

Autorización y artefactos derivados.

`ExecutionAuthorization` reifica el acuerdo que autoriza ejecuciones concretas, apoyándose en ODRL para permisos y restricciones, y `DerivedArtifact` representa la salida gobernable producida por una `it6:Run`. Esta separación evita confundir tres planos: política negociada, actividad de ejecución y resultado producido. DAIMO describe la cadena semántica que permite preguntar bajo

qué autorización se produjo un artefacto, no el mecanismo operativo que decide o ejecuta el acceso.

Evidencia y procedencia.

`AuditEvidence` registra evidencia verificable sobre una actividad mediante integridad, firmante y marca temporal; `CrossParticipantProvenanceRecord` agrupa actividades y entidades a través de contextos de participante para formar una narrativa de auditoría. PROV-O aporta el patrón de entidades, actividades y bundles; SPDX aporta el checksum estructurado. DAIMO no afirma cumplimiento automático ni validación externa de esas evidencias; proporciona las relaciones necesarias para que la evidencia pueda representarse y consultarse.

Tabla 7. Métricas principales del artefacto DAIMO.

Dimensión	Valor
Perfil lógico declarado	OWL 2 DL
Clases nativas de primer nivel	9
Subclases nativas	5
(ParticipantRole)	
Total clases DAIMO-nativas	14
Propiedades de objeto	29
(owl:ObjectProperty)	
Propiedades de datatype	8
(owl:DatatypeProperty)	
Propiedades funcionales	27
(owl:FunctionalProperty)	
Propiedades asimétricas	5
(owl:AsymmetricProperty)	
Pares inversos explícitos	5
(owl:inverseOf)	
rdfs:subClassOf a externos	8
rdfs:subPropertyOf a externos	6
Axioma de disyunción nombrado	1
sh:NodeShape totales	18
de completitud (una por clase DAIMO)	9
de conformidad sobre reutilizadas	3
invariantes	6
sh:SPARQLConstraint	
Preguntas de competencia	23
Pitfalls OOPS! (Crítico/Importante/Menor)	0 / 0 / 2

Tabla 8. Desglose OntoMetrics restringido al namespace DAIMO.

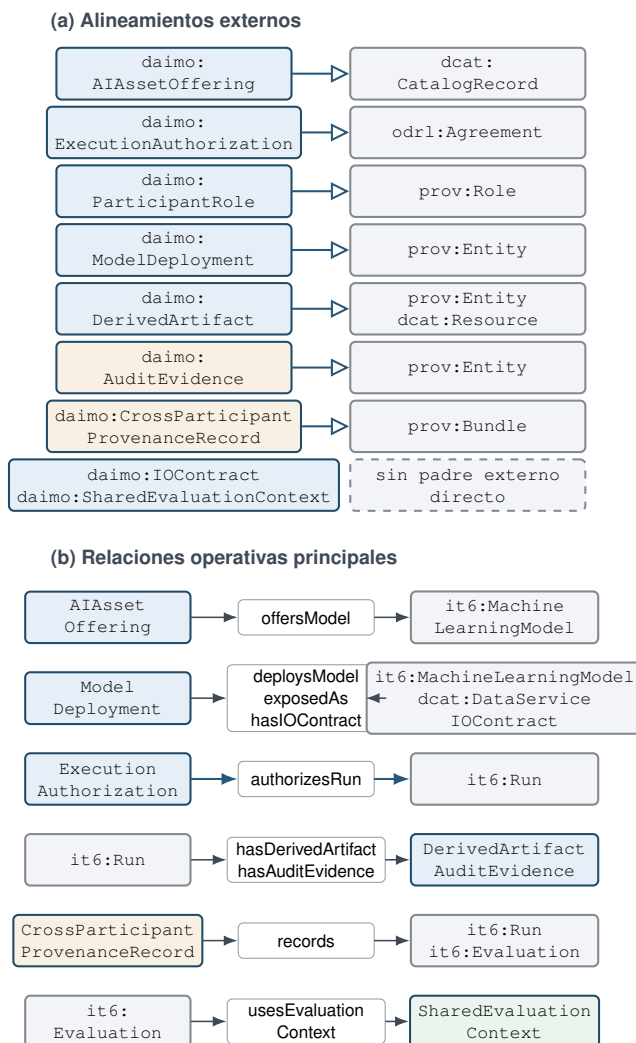
Tipo de axioma/declaración	Conteo
owl:Class	14
owl:ObjectProperty	29
owl:DatatypeProperty	8
rdfs:subClassOf	13
rdfs:subPropertyOf	6
rdfs:domain	37
rdfs:range	37
owl:FunctionalProperty	27
owl:AsymmetricProperty	5
owl:inverseOf	5
owl:AllDisjointClasses	1
owl:equivalentClass	0
owl:disjointWith explícitos	0

Evaluación compartida.

SharedEvaluationContext reifica las condiciones que hacen comparables dos evaluaciones: tarea, dataset, versión, protocolo y semilla. Las mediciones siguen siendo `it6:Evaluation` e `it6:EvaluationMeasure`; DAIMO sólo añade el contexto que evita interpretar una métrica como propiedad absoluta del modelo. Esta decisión mantiene fuera del núcleo una teoría universal de sesgo, equidad, explicabilidad o seguridad: esas dimensiones pueden expresarse como medidas contextualizadas o perfiles especializados cuando el caso de uso lo requiera.

Defensa compacta de alineamientos.

Los alineamientos externos siguen el mismo criterio de mínima duplicación. `AIAssetOffering` se alinea con `dcat:CatalogRecord`, no con `dcat:Dataset`,

**Figura 3.** Alineamientos externos y relaciones principales de DAIMO.

porque la oferta es el registro contextual que describe un modelo en un catálogo, no el modelo ni un dataset; la propiedad `offersModel` mantiene el enlace con `it6:MachineLearningModel`. `ExecutionAuthorization` se alinea con `odrl:Agreement` porque la autorización es el acuerdo efectivo que cubre ejecuciones, no un artefacto derivado de otro acuerdo. `ParticipantRole` se alinea con `prov:Role` con cautela: DAIMO añade alcance administrativo mediante el contexto de participante, pero conserva la lectura de rol como cualificación contextual de una participación.

Los demás alineamientos preservan la distinción entidad-actividad de PROV-O. `ModelDeployment` es una `prov:Entity`, no una actividad; `DerivedArtifact` es a la vez `prov:Entity` y `dcat:Resource`, porque es producido por una ejecución y puede catalogarse; `CrossParticipantProvenanceRecord` es un `prov:Bundle`, no una colección genérica; y `AuditEvidence` es una `prov:Entity`, no la actividad de auditar. `IOContract` y `SharedEvaluationContext` quedan sin padre externo directo por diseño: el primero no es un servicio ni

Tabla 9. Clases nativas, alineamientos axiomáticos y preguntas de competencia.

Clase	Alineamiento	Papel	CQs
daimo:AIAsset Offering	dcat:CatalogRecord	Registro de catálogo que publica un modelo como oferta gobernada en un dataspace; porta política ODRL mediante daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 subclases no disjuntas)	prov:Role	Rol asumido por un participante respecto a un activo de IA dentro de un contexto de dataspace.	R5
daimo:Model Deployment	prov:Entity	Instancia en ejecución o alojada del modelo, expuesta como dcat:DataService sobre una it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(sin alineamiento)	Contrato mínimo accionable: formatos de E/S, método de autenticación, esquemas opcionales.	R4, E1, G2
daimo:Execution Authorization	odrl:Agreement	Acuerdo ODRL derivado de una negociación DSP que autoriza ejecuciones concretas vía daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Salida gobernada y describible en catálogo producida por una ejecución bajo autorización.	E5, G4
daimo:CrossParticipant ProvenanceRecord	prov:Bundle	Agregación de actividades y entidades a través de contextos de participante para formar una narrativa de auditoría.	G4
daimo:AuditEvidence	prov:Entity	Registro de evidencia de cumplimiento con spdx:Checksum estructurado, firmante y marca temporal.	E4
daimo:SharedEvaluation Context	(sin alineamiento)	Reificación de las condiciones de comparabilidad: tarea, dataset, versión, protocolo y semilla aleatoria.	V1, V2, V3

un estándar, y el segundo no es una tarea ni una evaluación, sino la reificación de condiciones de comparabilidad.

Fundamentos axiomáticos

Una taxonomía por sí sola no fija el significado: sólo nombra los tipos. Detrás de la taxonomía de DAIMO hay una infraestructura axiomática que fija qué es cada clase, qué no es y cómo deben comportarse sus instancias. Tres estratos complementarios componen esta infraestructura: axiomas OWL sobre las clases y propiedades nativas, restricciones SHACL nodales sobre las instancias e invariantes SHACL-SPARQL entre clases. Los párrafos siguientes describen el primer estrato; los estratos SHACL aparecen en § como parte de la validación.

En el estrato OWL la disyunción entre los nueve tipos nativos de primer nivel se declara mediante el axioma nombrado daimo:TopLevelKindsDisjointness, instancia de owl:AllDisjointClasses. Convertir la disyunción en una entidad nombrada, en lugar de emitir pares anónimos owl:disjointWith, persigue tres beneficios. Primero, la disyunción puede referenciarse en documentación y reportes de validación. Segundo, un razonador puede citarla como causa específica cuando detecta una contradicción, facilitando el diagnóstico. Tercero, la disyunción entre las cinco subclases de ParticipantRole queda deliberadamente fuera del axioma: un mismo agente jurídico puede asumir varios roles simultáneamente (por ejemplo, proveedor y operador) sin generar inconsistencia lógica, reflejando la realidad de que los roles son propiedades contextuales del agente y no clases mutuamente exclusivas.

Al núcleo de disyunciones se añaden caracterizaciones sobre las propiedades DAIMO. Veintisiete propiedades se declaran funcionales (owl:FunctionalProperty) sobre aquellas cuyo dominio es una clase con *shape* nodal, de modo que la cardinalidad máxima queda axiomatizada simultáneamente en la capa lógica y en la capa SHACL correspondiente. Cinco propiedades clave (offersModel, deploysModel, authorizesRun, derivedFromRun y evidenceOf) se marcan como asimétricas, capturando formalmente que un

despliegue no es el modelo que despliega, que una autorización no coincide con la ejecución que autoriza, y así sucesivamente. Cinco pares inversos explícitos (authorizedBy ↔ authorizesRun, hasDeployment ↔ deploysModel, hasDerivedArtifact ↔ derivedFromRun, hasAuditEvidence ↔ evidenceOf, hasOffering ↔ offersModel) habilitan consultas inversas nativas sin obligar al publicador del grafo a materializar ambos sentidos.

El estrato SHACL se divide a su vez en dos familias. En primer lugar, cada clase DAIMO-nativa lleva asociada una *shape* nodal de completitud que prescribe qué propiedades son obligatorias en una instancia bien formada; estas nueve *shapes* son la garantía sintáctica de que un publicador ha suministrado todo lo necesario para que las invariantes semánticas operen sobre el grafo. En segundo lugar, tres *shapes* de conformidad (OfferInDAIMOShape, MachineLearningModelInDAIMOShape y RunInDAIMOShape) establecen qué propiedades de clases reutilizadas son obligatorias cuando un grafo declara adherirse al perfil DAIMO. La distinción entre ambas familias es deliberada: DAIMO no sobrescribe la ontología subyacente, porque eso fragmentaría el ecosistema, sino que declara su compromiso con un subconjunto concreto de propiedades sobre las clases externas, haciendo explícito el criterio de adhesión sin modificar las definiciones originales.

Por encima de las *shapes* se sitúan las seis invariantes SHACL-SPARQL entre clases (INV-1 a INV-6) que se examinan en detalle en §. Son el estrato que distingue a DAIMO de un simple perfil descriptivo: codifican reglas cuyo sentido no puede expresarse como restricción local a una sola *shape* porque involucran la coherencia entre varias clases simultáneamente. El Listado 1 muestra cómo se codifican los axiomas OWL para el caso concreto de ExecutionAuthorization, combinando la subclase con odrl:Agreement, la asimetría de authorizesRun, la funcionalidad de su inversa y la pertenencia al axioma de disyunción nombrado.

```
daimo:ExecutionAuthorization a owl:Class ;
```

```

rdfs:subClassOf odrl:Agreement ;
skos:definition "An ODRL agreement produced by
a DSP contract
negotiation that binds specific permissions
to run invocations..."@en .

daimo:authorizesRun a owl:ObjectProperty , owl:
AsymmetricProperty ;
rdfs:domain daimo:ExecutionAuthorization ;
rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:
FunctionalProperty ;
owl:inverseOf daimo:authorizesRun ;
rdfs:domain it6:Run ;
rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:
AllDisjointClasses ;
owl:members ( daimo:AIAssetOffering
daimo:ExecutionAuthorization
daimo:ModelDeployment
daimo:DerivedArtifact
daimo:IOContract
daimo:AuditEvidence
daimo:
CrossParticipantProvenanceRecord
daimo:SharedEvaluationContext
daimo:ParticipantRole ) .

```

Listing 1: Axiomas centrales alrededor de ExecutionAuthorization.

Superficie de reutilización. La Tabla 10 resume los vocabularios reutilizados y el uso concreto de cada uno. Tres observaciones son importantes. La primera es que las alineaciones hacia DCAT y ODRL son con los vocabularios estándar, no con entidades internas de ninguna implementación: este punto es relevante porque en la práctica de Eclipse EDC las dos capas se confunden con frecuencia. La segunda es que DAIMO mantiene hacia DSP una relación de mapeo informativo mediante `skos:closeMatch` y `skos:related`, no de subsunción: DSP es un protocolo de mensajería, no una ontología de activos, y subsumir clases DAIMO bajo constructos DSP introduciría tipados espurios. La tercera es que de toda la extensión EDC sólo `edc:ParticipantContext` aparece explícitamente en DAIMO, referenciado por su utilidad para anclar la procedencia agregada de un participante.

Tres subpropiedades que podrían parecer naturales no se declaran, y la ausencia merece documentación explícita. `daimo:authorizesRun` no se declara subpropiedad de `prov:used`; `daimo:grantedTo` no se declara subpropiedad de `prov:qualifiedAssociation`; `daimo:evidenceOf` no se declara subpropiedad de `prov:hadActivity`. En los tres casos, el alineamiento aparentemente intuitivo produciría, bajo razonamiento RDFS u OWL-RL, que las autorizaciones se tipasen como actividades PROV, los agentes receptores como asociaciones reificadas y las evidencias como objetos de influencia. Estos tipados contradicen el modelo conceptual del puente DAIMO. La verificación de implicación (§) se diseñó precisamente para detectar errores de este tipo antes de que contaminen el cierre inferido.

Evaluación

La evaluación distingue dos objetos: DAIMO como artefacto ontológico y la evaluación comparativa de modelos que DAIMO permite describir. El primer objeto se examina técnicamente mediante consistencia formal, cierre OWL-RL, conformidad SHACL, ausencia de pitfalls críticos, comportamiento de alineamientos y respuesta a preguntas de competencia. El segundo aparece sólo dentro del demostrador como contenido representado bajo un contexto compartido; no constituye un benchmark empírico de modelos reales. Por tanto, la evidencia reportada es ejecutable sobre el artefacto y sobre un grafo controlado, no validación externa por expertos ni despliegue productivo.

En términos de LOT, la adecuación al propósito se aborda de forma acotada mediante un grafo de instancia realista, consultas SPARQL ejecutables y pruebas negativas. Siguiendo la distinción habitual entre cobertura de requisitos, corrección formal, validación basada en restricciones y utilidad aplicada³³, preguntamos si el perfil representa el intercambio objetivo, si el artefacto OWL es formalmente coherente, si las restricciones SHACL discriminan grafos válidos e inválidos, y si las preguntas de competencia son ejecutables sobre el grafo materializado. Sobre esta distinción, el análisis sigue el patrón de cinco dimensiones de RePlanIT⁹: instanciación del escenario (§), verificación formal mediante razonamiento, comprobación de implicación y escaneo OOPS! (§), validación de instancias frente a SHACL (§), ejecución de las veintitrés preguntas de competencia (§) y verificación de los alineamientos de reutilización (§). Las cinco dimensiones son reproducibles con el paquete de evaluación que acompaña al artefacto.

Modelado del caso de uso

El escenario PIONERA introducido en § se instancia sobre DAIMO como demostrador controlado. El grafo resultante contiene 225 tripletas de datos que ejercen al menos una vez cada clase nativa de la ontología y sobre las que se ejecutan las veintitrés consultas de competencia reportadas en §. Todas las organizaciones, nombres de modelo, versiones de dataset, métricas y endpoints son anónimos o sintéticos, de modo que el demostrador instancia el caso PIONERA a nivel conceptual y no refleja ninguna instancia desplegada. El demostrador no es un estudio de despliegue, un *gold standard* ni una validación con catálogos reales de modelos; su función es examinar adecuación representacional y hacer ejecutables los artefactos de validación.

El conector de Empresa publica tres ofertas gobernadas en el catálogo federado de PIONERA: dos versiones de un mismo modelo de Movilidad de predicción de retraso y una variante de sólo lectura con política más estricta. Las ofertas comparten tarea y dominio, pero difieren en las condiciones ODRL asociadas. Esta parte del escenario ejercita la distinción entre el modelo como activo de IA descrito con MLDCAT-AP y la oferta como registro contextual de catálogo con condiciones de uso propias.

El modelo de Movilidad seleccionado se despliega sobre un entorno de cómputo descrito y se expone mediante dos interfaces de servicio: un endpoint REST con entrada JSON y autenticación por bearer token, y un endpoint gRPC

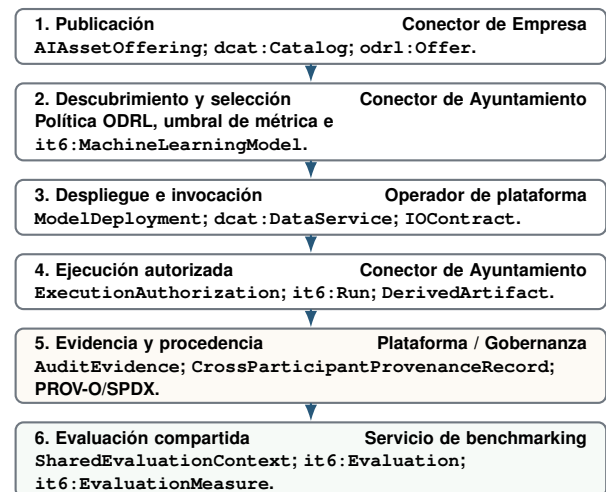
Tabla 10. Vocabularios reutilizados y papel de cada uno en DAIMO.

Vocabulario	Términos reutilizados	Uso en DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	Capa del activo de IA; DAIMO no redefine el modelo.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publicación y descubrimiento.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Política, oferta y acuerdo.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Procedencia y roles.
SPDX (spdx:)	Checksum, algorithm, checksumValue	Integridad de evidencia de auditoría.
DSP (dspace:)	ContractOffer, ContractNegotiation, TransferProcess	Mapeos informativos skos:closeMatch y skos:related; no subsunción.
EDC (edc:)	ParticipantContext	Única extensión específica de EDC referenciada.

con protobuf y autenticación mTLS. El uso multi-endpoint ejercita la decisión de modelado según la cual un mismo despliegue puede exponer varios servicios y contratos, en lugar de forzar un despliegue distinto por endpoint.

El conector de Ayuntamiento obtiene una autorización de ejecución con fechas de vigencia explícitas, invoca el modelo y recibe un artefacto de predicción derivado. La ejecución se vincula con evidencia de auditoría que contiene registro de integridad, firmante y marca temporal, de modo que un actor de gobernanza pueda reconstruir cómo se produjo el artefacto y bajo qué autorización. Un registro de procedencia cruzada agrega la cadena a través de los contextos de Empresa, Ayuntamiento y plataforma. Un servicio de benchmarking de PIONERA publica dos resultados sintéticos de evaluación bajo el mismo benchmark, protocolo y semilla aleatoria, permitiendo representar una comparación entre dos versiones del modelo bajo condiciones compartidas.

La Figura 4 sintetiza el demostrador como flujo por fases. Las fases se leen de arriba abajo, desde publicación hasta evaluación compartida, y cada una conserva sólo los constructos necesarios para explicar el papel representacional de DAIMO. Los detalles finos de propiedades quedan en el texto y en la trazabilidad de consultas.

**Figura 4.** Vista por fases del demostrador multi-participante acotado.

Verificación formal

El razonador OWL 2 DL HermiT, invocado mediante `owlready2`, reporta que la unión del núcleo, los alineamientos y el grafo de ejemplo es lógicamente consistente y que no existen clases insatisfactibles; el tiempo de razonamiento reportado es de 1,25 segundos. La materialización OWL-RL (biblioteca `owlrl`) deriva 1170 tripletas adicionales sobre 818 iniciales (1988 totales) en 0,58 segundos, sin producir individuos tipados como `owl:Nothing` ni subclases insatisfactibles. Ambos razonadores coinciden, con HermiT como validador de la corrección lógica y OWL-RL como materializador pragmático para las consultas que requieren inferencia por subpropiedad o subclase.

La consistencia lógica es condición necesaria pero no suficiente para la corrección semántica de una ontología de integración. Una axiomatización puede ser perfectamente

consistente y, al mismo tiempo, implicar tipados no intencionados si no declara disyunción explícita con la clase adecuada o si adopta una subpropiedad cuyo rango fuerza una subsunción indeseada. Por eso DAIMO implementa adicionalmente una *verificación de implicación* sobre el cierre OWL-RL: para cada clase nativa, la comprobación enumera todas las superclases inferidas y las contrasta con una lista de superclases prohibidas. Por ejemplo, `AuditEvidence` no debe ser inferida como `prov:Activity`, porque es una cosa producida por actividades, no una actividad. El reporte cubre las catorce clases nativas (nueve de primer nivel más cinco subclases de `ParticipantRole`) y no emite ninguna advertencia. Esta comprobación es complementaria a la de consistencia y captura errores que un razonador DL acepta porque son lógicamente consistentes aunque semánticamente incorrectos.

El escáner OOPS!³⁴, aplicado sobre la unión del núcleo y los alineamientos, reporta cero pitfalls Críticos, cero Importantes y dos Menores. El Menor P13 (“relaciones inversas no declaradas”) afecta a 34 elementos, y corresponde a propiedades para las que no existe una inversa semánticamente significativa (por ejemplo, `daimo:integrityHash`: nombrar una inversa “tiene hash” no añade capacidad de consulta). El Menor P04 (“elementos no conectados”) afecta a siete elementos que son clases externas declaradas localmente para permitir que el razonador las interprete sin exigir que el archivo las defina completamente. Ambos Menores se interpretan como efectos esperables de un perfil de integración *reuse-first*, no como evidencia de pitfalls críticos o importantes.

Las métricas de calidad estructural se interpretan como señales diagnósticas, no como criterios absolutos de aceptación. Conteos como profundidad de jerarquía, amplitud taxonómica, conectividad, equivalencias explícitas o elementos aislados ayudan a localizar zonas que merecen inspección, pero no dictan por sí mismos una corrección automática. En OWL, un ciclo de `rdfs:subClassOf` puede expresar equivalencia implícita antes que un error; una `owl:equivalentClass` puede ser un alineamiento legítimo; y una disyunción sólo debe añadirse cuando el compromiso ontológico es verdadero en todo modelo posible del dominio. DAIMO usa por ello OOPS!, HerMiT, OWL-RL, SHACL, pruebas negativas y preguntas de competencia como capas complementarias: cada una observa un tipo de riesgo distinto y ninguna sustituye el juicio de modelado.

Validación SHACL: estructural, invariantes y banco negativo

DAIMO superpone tres familias de *shapes* SHACL, cada una respondiendo a una pregunta de validación distinta. Las *shapes* estructurales preguntan si toda instancia DAIMO-nativa porta las propiedades obligatorias de su clase. Las *shapes* de conformidad preguntan si las clases reutilizadas externas satisfacen las obligaciones adicionales que el perfil les impone. Las invariantes SPARQL preguntan si las relaciones entre oferta, política, despliegue, autorización, ejecución, artefacto derivado y evidencia siguen siendo coherentes. Juntas, las tres familias llevan la validación de lo sintáctico a lo semántico: de “¿está esta instancia bien

formada?” a “¿es este grafo internamente consistente?”. La capa SHACL funciona así como contrato de datos del perfil DAIMO; no introduce una ontología paralela ni exige trazabilidad, formatos de modelo o métricas universales que no estén justificadas por las preguntas de competencia.

En términos de implementación, las restricciones se expresan como `sh:NodeShape` dirigidas a clases DAIMO o clases reutilizadas bajo el perfil, y como `sh:PropertyShape` internas que fijan cardinalidades, tipos de dato, clases esperadas, patrones o enumeraciones cerradas. Como patrón de consumo, un publicador podría validar metadatos RDF o JSON-LD de un modelo contra la capa SHACL y obtener un `sh:ValidationReport` que indique si el grafo conforma o qué propiedad incumple el perfil. Este patrón explica el uso previsto de las *shapes*; no se presenta como un conector productivo ya desplegado ni como una validación de la ontología TBox en sí misma.

Las nueve *shapes* estructurales imponen completitud mínima por clase; el grafo positivo de 225 tripletas satisface `conforms=True`. Las tres *shapes* de conformidad añaden obligaciones operativas sobre clases reutilizadas: `OfferInDAIMOShape` exige que toda `odrl:Offer` utilizada en DAIMO declare `odrl:assigner` y `odrl:target` (a nivel de política o por permiso, mediante el conectivo `sh:or`); `MachineLearningModelInDAIMOShape` exige política ODRL, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`. Adicionalmente, `sh:in` restringe los valores de `daimo:authMethod` a un vocabulario cerrado de nueve tokens (`none`, `api-key`, `basic`, `bearer`, `oauth2-bearer`, `oauth2-client-credentials`, `mtls`, `jwt`, `dataspace-token`), y `sh:pattern` restringe `daimo:protocol` a un patrón que acepta los nombres habituales de protocolo de evaluación (`holdout`, `stratified-holdout`, `leave-one-out`, `temporal-split`, `<n>-fold-cv`, `stratified-<n>-fold-cv`, `bootstrap-<n>`).

Sobre este estrato estructural se sitúan las seis invariantes SHACL-SPARQL entre clases, codificadas mediante `sh:SPARQLConstraint`, que son el elemento diferenciador de DAIMO frente a un perfil puramente descriptivo. La invariante INV-1 garantiza la consistencia de la cadena de autorización de derivación: todo `DerivedArtifact` debe portar una propiedad `daimo:underAuthorization` cuya autorización cubra precisamente la ejecución de la que el artefacto se deriva. La INV-2 impone que el agente asociado a una ejecución mediante `prov:wasAssociatedWith` aparezca también como `daimo:grantedTo` en alguna de las autorizaciones que cubren esa ejecución, haciendo explícito que el actor registrado en PROV y el beneficiario de la autorización ODRL deben ser la misma entidad. La INV-3 verifica la coherencia del plano de datos: el `dcat:DataService` expuesto por un `ModelDeployment` debe servir exactamente el modelo que el despliegue declara desplegar, eliminando la posibilidad de que un endpoint anuncie un modelo pero sirva otro. La INV-4 exige que el `daimo:expiresAt` de cada `ExecutionAuthorization` sea estrictamente posterior al `prov:startedAtTime` de toda ejecución

que la autorización cubre, reflejando que una autorización no puede justificar ejecuciones que comenzaron tras su caducidad. La INV-5 requiere que el modelo referenciado por `daimo:offersModel` en una `AIAssetOffering` aparezca como `odrl:target` de la política asociada, a nivel de política o en alguno de sus permisos; si la política no gobierna el modelo registrado, la cadena de gobernanza está rota aunque ambos recursos existan. La INV-6 cierra el mismo triángulo por el lado del proveedor: el `daimo:offeredBy` de toda oferta debe coincidir con el `odrl:assigner` de su política, impidiendo inconsistencias entre la atribución del registro de catálogo y la autoría declarada en la oferta ODRL. El grafo positivo conforma las seis invariantes sin advertencias. El Listado 2 muestra la codificación concreta de INV-5, ilustrando el patrón `FILTER NOT EXISTS` que captura la disyunción entre política global y permiso individual.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape
;
sh:targetClass daimo:AIAssetOffering ;
sh:sparql [
  a sh:SPARQLConstraint ;
  sh:message "INV-5 violation: offering's
    offersModel does not
      appear as odrl:target of the
        attached policy."@en ;
  sh:prefixes daimo:_invariantPrefixes ;
  sh:select """
    SELECT $this ?model ?policy WHERE {
      $this daimo:offersModel ?model ;
      daimo:hasOfferPolicy ?policy .
      FILTER NOT EXISTS { ?policy odrl:
        target ?model }
      FILTER NOT EXISTS { ?policy odrl:
        permission/odrl:target ?model }
    }
    """ ;
]
```

Listing 2: Invariante INV-5 como `sh:SPARQLConstraint`.

Las *shapes* que conforman sobre el grafo positivo son necesarias pero no suficientes para evaluar si las restricciones son semánticamente discriminantes: una restricción trivialmente satisfacible sobre cualquier grafo pasaría la validación positiva sin aportar evidencia adicional. Por eso DAIMO incorpora un grafo negativo de 118 tripletas que codifica seis casos deliberadamente inválidos —`bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering` y `bad:INV6-offering`—, cada uno focalizado en una invariante. La prueba carga la ontología, los *shapes* y el grafo negativo, ejecuta SHACL y verifica dos condiciones: que la validación falle globalmente y que cada nodo focalizado esperado aparezca en el informe con el mensaje específico de la invariante violada. Las seis invariantes se disparan correctamente sobre su violación correspondiente. Esta evidencia, complementaria a la validación positiva, respalda que las invariantes capturan restricciones interclase no triviales del perfil.

Preguntas de competencia

Las veintitrés preguntas de competencia se expresan como consultas SPARQL y se ejecutan sobre el cierre OWL-RL materializado del grafo positivo. La tabla siguiente resume la cobertura por categoría y conserva los conteos observados; la trazabilidad fila a fila queda en la Tabla 15, en el Apéndice. Las 23/23 consultas devuelven al menos una fila, lo que verifica que el grafo de demostración contiene la información necesaria para responder a cada pregunta con las primitivas que DAIMO declara; no evalúa la calidad empírica de los modelos representados.

Dos resultados conviene señalar. Las consultas CQ-R2 y CQ-E1 dependen de la relación `subPropertyOf` entre `daimo:offersModel` y `foaf:primaryTopic` y no devuelven filas sobre un motor SPARQL puro sin razonamiento; CQ-R5 recorre explícitamente la jerarquía de roles. La dependencia es documentada, no un defecto: alinear `offersModel` bajo `foaf:primaryTopic` hace las ofertas descubribles mediante la propiedad DCAT estándar, beneficio que sólo se materializa bajo inferencia. Un consumidor que necesite ejecutar esas consultas sobre un endpoint puro puede materializar el cierre previamente o reescribir las consultas enumerando ambas propiedades. El segundo resultado es que CQ-G2 devuelve dos filas —una por endpoint del despliegue multi-protocolo REST y gRPC—, lo que confirma que el modelado de múltiples servicios por despliegue es representable y directamente consultable desde SPARQL.

Dos fragmentos ilustran el patrón de formalización. El Listado 3 muestra CQ-G3: la consulta recupera la autorización que cubre una ejecución concreta, el beneficiario autorizado y la fecha de expiración. La alineación `ExecutionAuthorization` $\sqsubseteq$ `odrl:Agreement` hace que ese recurso sea interpretable como acuerdo ODRL sin duplicar propiedades de contrato en la consulta. El Listado 4 muestra CQ-V3: dos evaluaciones se comparan sólo porque comparten el mismo `SharedEvaluationContext`; el ordenamiento por valor de métrica es significativo dentro de ese contexto y no como ranking global independiente del dataset, protocolo o semilla.

```
SELECT ?auth ?grantee ?expires WHERE {
  ?auth a daimo:ExecutionAuthorization ;
  daimo:authorizesRun
    <https://example.org/daimo-scenario/run
      -2026-04-20-legs> ;
  daimo:grantedTo ?grantee ;
  daimo:expiresAt ?expires .
}
```

Listing 3: CQ-G3: autorización de una ejecución concreta.

```
SELECT ?model ?value WHERE {
  ?eval a it6:Evaluation ;
  it6:evaluates ?model ;
  daimo:usesEvaluationContext
    <https://example.org/daimo-scenario/
      evalctx-climatebench-v1-2026-1-holdout
    > ;
  it6:hasEvaluationMeasure ?m .
  ?m it6:hasValue ?value .
}
ORDER BY DESC(?value)
```

Tabla 11. Cobertura de las 23 preguntas de competencia sobre el grafo positivo cerrado con OWL-RL.

Categoría	Consultas (filas)	N.	Operación semántica o SPARQL relevante
Registro y publicación (R)	R1(3) R2(1) R3(1) R4(2) R5(2)	5	R2 (cadena <code>subPropertyOf</code>), R5 (ruta <code>subClassOf</code>)
Descubrimiento y selección (D)	D1(3) D2(2) D3(4) D4(1)	4	D2 usa negación como fallo; D4 aplica filtro numérico. D1 y D3 son consultas directas.
Ejecución y auditabilidad (E)	E1(4) E2(1) E3(2) E4(1) E5(1)	5	E1 (<code>offersModel $\sqsubseteq$ foaf:primaryTopic</code>); E2 usa unión acuerdo-ejecución.
Evaluación y reproducibilidad (V)	V1(1) V2(1) V3(2) V4(1) V5(4)	5	V2 usa agregación <code>MAX</code> ; V3 usa ordenamiento.
Puente de gobernanza (G)	G1(1) G2(2) G3(1) G4(1)	4	G3 ejerce el puente DAIMO-ODRL; G4 usa <code>GROUP_CONCAT</code> .

Listing 4: CQ-V3: ranking de modelos bajo un contexto compartido.

Reutilización verificada. La reutilización se verifica directamente sobre el módulo de alineamientos. Ocho `rdfs:subClassOf` y cinco `rdfs:subPropertyOf`, listados en la Tabla 9, conectan DAIMO con sus cinco vocabularios parentales (DCAT, MLDCAT-AP, ODRL, PROV-O y FOAF); los alineamientos de clase se visualizan en la Figura 3. La sexta subpropiedad DAIMO hacia un vocabulario externo, `hasOffering  $\sqsubseteq$  foaf:isPrimaryTopicOf`, se declara en el núcleo porque implementa la navegación inversa de `offersModel`. La verificación opera en dos dimensiones.

La primera es *estructural*: toda clase nativa que podría haberse definido aisladamente se sitúa bajo un padre externo siempre que dicho padre exista, y toda propiedad nueva extiende una propiedad reutilizada siempre que ésta ya cargue el significado previsto. No se introduce ninguna clase por razones estilísticas, y las clases `IOContract` y `SharedEvaluationContext` quedan sin padre externo sólo porque ningún padre con semántica veraz existe (§).

La segunda es *semántica*: los alineamientos no deben producir inferencias no intencionadas. La verificación de implicación de § inspecciona el cierre OWL-RL de cada alineamiento y confirma que ninguno produce una superclase semánticamente incorrecta sobre ninguna clase nativa. El razonador reporta cero advertencias sobre las catorce clases. Tres alineamientos de subpropiedad hacia PROV-O se excluyeron deliberadamente del módulo por el mismo motivo: cada uno habría producido un tipado prohibido bajo cierre RDFS (§).

Cobertura de requisitos no funcionales

Los requisitos no funcionales se evalúan sobre el artefacto ontológico y sus flujos de validación, no sobre una plataforma de ejecución federada. En particular, la escalabilidad se interpreta como crecimiento del número de instancias de intercambio gobernado en el ABox; el núcleo DAIMO permanece estable en catorce clases nativas. Por tanto, el resultado no mide concurrencia, latencia de un conector ni rendimiento productivo de inferencia en línea, sino tratabilidad reproducible de grafos DAIMO crecientes bajo las operaciones que el perfil exige: parseo RDF, materialización OWL-RL, validación SHACL y consultas SPARQL.

La cobertura no funcional se resume antes del benchmark para separar el tipo de evidencia disponible de los riesgos

que quedan fuera del artefacto. La Tabla 12 mantiene esta distinción entre validación reproducible, interoperabilidad potencial y ausencia de afirmaciones productivas.

El benchmark se presenta como evidencia acotada de tratabilidad, no como prueba de rendimiento operativo. La Tabla 13 reporta los tiempos y conteos de tripletes obtenidos para grafos de 100 y 1 000 unidades sintéticas de intercambio, junto con el veredicto de conformidad SHACL en cada caso. Las dos filas pretenden mostrar una tendencia de crecimiento bajo un protocolo controlado, no certificar rendimiento sobre una carga de producción. También aíslan el coste de cada etapa —parseo, cierre OWL-RL, validación SHACL y respuesta SPARQL— de modo que el factor dominante quede visible. Las cifras son reproducibles desde la suite de validación publicada bajo la semilla aleatoria fija usada para generar los grafos.

Ambas configuraciones se mantienen conformes con SHACL, y la respuesta a consultas permanece muy por debajo de un segundo incluso con 1 000 unidades, lo que indica que el perfil no introduce un coste patológico de consulta. Como cabía esperar, la validación SHACL es la etapa dominante y crece más rápido que el parseo o el cierre, de modo que es el componente a vigilar cuando los grafos crecen. Una ejecución mayor de 10 000 unidades se deja deliberadamente como trabajo futuro, porque requeriría ejecución real y no la extrapolación aquí reportada.

Discusión

DAIMO aporta un perfil de integración para representar modelos de IA como activos gobernados en espacios de datos. Su contribución es semántica y de gobernanza: conecta catálogo, política, despliegue, ejecución, evidencia y evaluación sin estandarizar pesos, arquitecturas internas ni mecanismos de inferencia. Esta posición lo aproxima a ontologías recientes de Semantic Web Journal por su énfasis en reutilización, implementación ejecutable y preguntas de competencia, pero aplica ese patrón al puente entre catálogo de IA y plano de intercambio.

La relación con los vocabularios reutilizados es deliberadamente complementaria. MLDCAT-AP sigue siendo la fuente del modelo mediante `it6:MachineLearningModel`; DCAT aporta catálogos y servicios; ODRL expresa ofertas y acuerdos; PROV-O estructura procedencia; DSP describe el protocolo de negociación; y EDC se trata como posible implementación, no como ontología de dominio. DAIMO añade únicamente relaciones puente donde esa combinación no permite consultar la cadena gobernada completa, por ejemplo oferta,

Tabla 12. Cobertura de requisitos no funcionales y riesgo residual.

Requisito	Evidencia reproducible	Estado y riesgo residual
Consistencia lógica	HermiT reporta consistencia y cero clases insatisfactibles; OWL-RL materializa 1 170 tripletas sin <code>owl:Nothing</code> ; la verificación de implicaciones inspecciona catorce clases nativas con cero advertencias.	Evidencia fuerte a nivel de artefacto. La consistencia se complementa con pruebas negativas SHACL para evitar restricciones triviales.
Interoperabilidad	El perfil usa RDF/OWL 2 DL, SHACL y SPARQL; reutiliza DCAT, MLDCAT-AP, ODRL, PROV-O, FOAF, SKOS y SPDX mediante IRIs externos y alineamientos axiomáticos.	Evidencia fuerte a nivel de artefacto. La interoperabilidad operacional dependerá de la publicación y consumo efectivos en infraestructuras de terceros.
Escalabilidad	El benchmark genera grafos sintéticos conformes de 100 y 1 000 unidades de intercambio; ambos conforman SHACL y responden las consultas de control.	Cobertura medida y acotada. SHACL es el coste dominante; no se afirma rendimiento productivo ni resultado de 10 000 unidades sin ejecutarlo.
Extensibilidad	El núcleo mantiene alcance mínimo y versionado SemVer; las extensiones se orientan a perfiles complementarios, por ejemplo <code>daimo-discovery</code> , sin absorber lógica de ranking o ejecución.	Evidencia fuerte a nivel de artefacto para evolución controlada. Nuevos conceptos deben justificar que no existen en vocabularios reutilizados.
Modularidad	Núcleo, alineamientos, <i>shapes</i> , ejemplos, consultas, pruebas negativas y reportes están separados; cada módulo puede ejecutarse o revisarse de forma independiente.	Evidencia fuerte a nivel de artefacto. La independencia modular reduce el riesgo de romper el núcleo al añadir perfiles externos.
Documentación	Cada término incluye etiquetas y definiciones SKOS; la documentación suplementaria traza clases, propiedades, CQs, invariantes y comandos de reproducción.	Evidencia fuerte a nivel de artefacto para revisión local y reproducción. La calidad editorial de la documentación pública deberá mantenerse sincronizada con futuras versiones del perfil.

Tabla 13. Benchmark sintético de escalabilidad acotada.

Unidades	Tripletas datos	Tripletas cierre	Parseo (s)	OWL-RL (s)	SHACL (s)	SPARQL (s)	Conforma
100	8 053	16 248	0,468	5,478	10,374	0,080	Sí
1 000	80 053	147 648	4,660	53,672	135,010	0,356	Sí

despliegue, autorización de ejecución y procedencia entre contextos administrativos.

Las limitaciones semánticas delimitan el alcance del perfil. DAIMO no es un catálogo de catálogos ni un motor de consulta federada. El `IOContract` captura formato, autenticación y referencias opcionales a esquemas, pero no resuelve la compatibilidad semántica fina entre entradas y salidas. Del mismo modo, DAIMO describe ejecuciones autorizadas y auditables, pero no implementa runtime, enforcement federado, orquestación de contenedores ni mecanismos TEE. Esas decisiones pertenecen a plataformas o perfiles especializados.

También quedan fuera del núcleo varias extensiones de dominio. `SharedEvaluationContext` permite comparar resultados bajo una misma tarea, dataset, versión, protocolo y semilla, pero no define una teoría universal de sesgo, equidad, explicabilidad o seguridad. Las restricciones heredadas desde datasets de entrenamiento hacia modelos o artefactos derivados tampoco se axiomatizan como regla general: dependen de contrato, licencia, jurisdicción y técnica de entrenamiento. Cuando sean necesarias, deben expresarse mediante perfiles SHACL u ODRL específicos del espacio de datos.

Algunos compromisos de integración producen efectos que los consumidores deben conocer. La propiedad `daimo:records` puede inferir evaluaciones MLDCAT-AP como actividades PROV cuando se agrupan en un `CrossParticipantProvenanceRecord`; este efecto es útil para auditoría, pero debe documentarse en integraciones que combinen DAIMO, MLDCAT-AP y PROV-O. DAIMO tampoco se ancla a una ontología de nivel superior,

porque se presenta como perfil de integración y no como ontología de referencia.

Las limitaciones de validación son igualmente importantes. La evidencia reportada es validación ejecutable del artefacto y del grafo demostrador: consistencia, cierre OWL-RL, SHACL, pruebas negativas, OOPS!, CQs y escalabilidad sintética acotada. No sustituye una revisión externa por expertos, no prueba despliegue productivo y no constituye cumplimiento automático del AI Act. Además, los resultados exactos dependen de condiciones de razonamiento documentadas, en particular inferencia RDFS durante SHACL y cierre OWL-RL antes de ejecutar las consultas de competencia.

El cierre práctico es, por tanto, una agenda de validación posterior: instanciar DAIMO sobre catálogos MLDCAT-AP reales, probar su consumo con conectores de espacios de datos, revisar el perfil con expertos de dominio y contrastar casos de alto riesgo frente a requisitos regulatorios concretos. Ninguna de esas tareas se reclama como completada en este artículo; todas son pasos necesarios para pasar de validación ejecutable del artefacto a validación externa y despliegue real.

La integración futura debe mantener esta frontera. Aplicaciones de descubrimiento, ranking, recomendación, ejecución federada o interfaz de usuario pueden consumir grafos DAIMO, pero no deben absorberse en el núcleo. El detalle de familias complementarias y rutas de extensión se conserva en el Apéndice para no alargar el cierre del artículo.

Disponibilidad y sostenibilidad

DAIMO se acompaña de los artefactos necesarios para inspeccionar y reproducir la evaluación: módulos OWL en Turtle, alineamientos, *shapes* SHACL, grafo de

demostración, consultas de competencia, banco negativo y reportes generados. La ontología y su documentación se declaran bajo licencia CC-BY 4.0, mientras que el código de validación se declara bajo Apache 2.0. La reproducibilidad no depende de un endpoint productivo.

La sostenibilidad sigue la Fase 4 de LOT mediante IRIs versionadas, metadatos Dublin Core, política SemVer y separación entre núcleo, alineamientos y capa SHACL. Estos elementos apoyan disponibilidad y reutilización del artefacto; no se presentan como evidencia de adopción, despliegue real o validación externa.

Conclusiones

DAIMO propone una contribución *reuse-first*: reutiliza MLDCAT-AP, DCAT, ODRL, PROV-O, DSP/EDC, FOAF, SKOS y SPDX, y añade sólo clases puente para articular catálogo, política, despliegue, ejecución, evidencia y evaluación en espacios de datos. El modelo de IA no se redefine; se mantiene como `it6:MachineLearningModel`.

La evidencia presentada es reproducible y acotada: consistencia OWL, cierre OWL-RL, conformidad SHACL, pruebas negativas, escaneo OOPS!, 23 preguntas de competencia y un benchmark sintético de escalabilidad. Esta evidencia respalda la coherencia del artefacto y del demostrador controlado, pero no equivale a validación externa, adopción institucional ni despliegue productivo.

El trabajo futuro debe concentrarse en validar DAIMO fuera del entorno controlado: instanciación contra catálogos MLDCAT-AP reales, revisión cualitativa con expertos, integración piloto con conectores de espacios de datos, refinamiento del `IOContract`, separación más fina entre tipo de rol y asignación de rol, extensión del `SharedEvaluationContext` a protocolos heterogéneos y perfiles complementarios de reproducibilidad, monitorización, seguridad y cumplimiento.

Referencias

- Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspace: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33.
- Albertoni, A. et al. (2020) *Data Catalog Vocabulary (DCAT) — Version 2*. W3C Recommendation.
- Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) *PROV-O: The PROV Ontology*. W3C Recommendation.
- Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) *ODRL Information Model 2.2*. W3C Recommendation.
- Knublauch, H. and Kontokostas, D. (eds.) (2017) *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- Mitchell, M. et al. (2019) Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity. *IBM Research Report*.
- Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460.
- Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*.
- Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*.
- Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. *Semantic Web*.
- Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. *Semantic Web*, 14:585–610.
- Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554.
- Ferranti, N. et al. (in press) Formalizing and Validating Wikidata's Property Constraints using SHACL and SPARQL. *Semantic Web*.
- Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*.
- Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. *Semantic Web*.
- Otto, B. et al. (2019) Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199.
- Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46.
- Geburu, T. et al. (2021) Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
- Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37.
- Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. *KDD*.
- Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11).
- Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
- Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. *ISWC*.
- Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. *ICDMW*.
- Keet, C.M. et al. (2015) The data mining optimization ontology. *Web Semantics*, 32.
- Joachimaki, M.P. et al. (2024) The Artificial Intelligence Ontology: LLM-assisted construction of AI concept hierarchies. *Applied Ontology*.
- Blagec, K., Barbosa-Silva, A., Ott, S. and Samwald, M. (2022) A curated, ontology-based, large-scale knowledge graph of artificial intelligence tasks and benchmarks. *Scientific Data*, 9:322.
- Novacek, J., Ahari, A., Müller, T., Reiter, S., Viehl, A. and Bringmann, O. (2024) Ontology-Supported AI Model and Dataset Management. *IEEE 22nd International Conference on Industrial Informatics (INDIN)*, 1–6.
- Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. *WORKS*.
- Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. *ICDE*

Workshops.

32. Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755.
33. Brank, J., Grobelnik, M. and Mladenić, D. (2005) A survey of ontology evaluation techniques. *Conference on Data Mining and Data Warehouses*.
34. Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (Ontology Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34.
35. SEMIC (2025) *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission.
36. SEMIC (2026) *MLDCAT-AP 3.1.0: Machine Learning Data Catalogue Application Profile*. European Commission, Candidate Recommendation, 13 May 2026.
37. International Data Spaces Association (2024) *Dataspace Protocol (DSP)*. W3C Community Draft.
38. Eclipse Dataspace Protocol (2025) *Dataspace Protocol Release 2025-1*. Eclipse Dataspace Working Group.
39. Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation.
40. Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. *ISWC*.
41. European Union (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union.

Familias complementarias y perfiles futuros

La discusión principal resume la frontera entre núcleo DAIMO y perfiles especializados. La tabla siguiente conserva el detalle de esa decisión de alcance: qué cubre el perfil actual, qué queda fuera del núcleo y qué debería abordarse como extensión futura sin modificar las catorce clases nativas.

La Tabla 14 hace auditable la decisión de alcance: la columna central muestra que el núcleo actual ya cubre lo que exigen el descubrimiento y la invocación, mientras que la columna derecha nombra el perfil que alojaría cada preocupación adicional. El patrón constante es que las extensiones se añaden como perfiles separados versionados con SemVer y no como nuevas clases núcleo, lo que mantiene estables las catorce clases nativas. Este es el mecanismo por el que DAIMO puede evolucionar hacia reproducibilidad, linaje, monitorización o cumplimiento sin fragmentar su columna vertebral de gobernanza.

Preguntas de competencia

R — Registro y publicación (5)

1. CQ-R1: ¿Qué modelos ha publicado un proveedor dado como activos de catálogo gobernados?
2. CQ-R2: Dado un modelo, ¿qué oferta lo registró, por qué proveedor y en qué fecha? (requiere cadena `subPropertyOf`)
3. CQ-R3: ¿Qué licencia y política de uso aplican a un modelo publicado?

4. CQ-R4: ¿Qué contrato de entrada/salida se declara para la interfaz de invocación?
5. CQ-R5: Para cada oferta, ¿qué subclase concreta de `ParticipantRole` tiene el agente proveedor? (requiere `subClassOf+`)

D — Descubrimiento y selección (4)

1. CQ-D1: ¿Qué modelos resuelven una tarea dada en un dominio dado?
2. CQ-D2: ¿Qué modelos son utilizables bajo un patrón de política concreto (por ejemplo uso no comercial)?
3. CQ-D3: ¿Qué modelos exponen un endpoint y qué método de autenticación requiere cada uno?
4. CQ-D4: ¿Qué modelos alcanzan un umbral mínimo de métrica bajo un contexto de evaluación dado?

E — Ejecución y auditabilidad (5)

1. CQ-E1: Dada una oferta, ¿qué endpoint, método de autenticación y contrato de entrada/salida aplican al modelo subyacente? (requiere entailment)
2. CQ-E2: Para un despliegue dado, ¿qué ejecuciones, por qué agentes y bajo qué autorización?
3. CQ-E3: Para una ejecución dada, ¿qué implementación, algoritmo, *flow* e infraestructura?
4. CQ-E4: ¿Qué evidencia de auditoría (hash, firmante, marca temporal) respalda una ejecución?
5. CQ-E5: ¿Qué artefactos derivados produjo una ejecución y bajo qué autorización?

V — Evaluación y reproducibilidad (5)

1. CQ-V1: ¿Qué contexto compartido (dataset, versión, protocolo, semilla) usa una evaluación?
2. CQ-V2: Bajo un contexto compartido, ¿qué modelo alcanza el mayor valor de métrica?
3. CQ-V3: ¿Cómo se ordenan dos o más modelos bajo el mismo contexto y métrica?
4. CQ-V4: Para un modelo dado, ¿sobre qué benchmark suites ha sido evaluado?
5. CQ-V5: ¿Qué artefactos de reproducibilidad (flow, cuaderno, tabla de resultados, checksum) respaldan una evaluación?

G — Puente de gobernanza (4, DAIMO-nativas)

1. CQ-G1: ¿Qué ofertas del catálogo federado incluyen un modelo dado?
2. CQ-G2: ¿Qué despliegues sirven un modelo dado, sobre qué infraestructura y con qué contrato de entrada/salida?
3. CQ-G3: ¿Qué autorización, y el acuerdo del que deriva, permitió una ejecución específica?
4. CQ-G4: A través de contextos de participante, ¿cuál es el bundle de procedencia completo para un artefacto derivado?

Tabla 14. Familias complementarias y ruta de evolución controlada.

Familia	Cobertura en DAIMO actual	Ruta de extensión controlada
Disponibilidad del recurso	Endpoint mediante <code>ModelDeployment</code> , <code>dcat:DataService</code> y <code>dcat:endpointURL</code> ; política y licencia mediante <code>ODRL</code> y <code>dct:license</code> ; distribuciones descargables mediante <code>DCAT</code> cuando el modelo se publica como archivo; proveedor mediante roles DAIMO y contexto de participante. <code>dct:publisher</code> queda reservado para el mantenedor del registro o catálogo cuando proceda.	Mantener en el núcleo sólo la disponibilidad necesaria para descubrimiento e invocación. Formatos de artefacto como <code>ONNX</code> o <code>PyTorch</code> se expresan con <code>dct:format/dcat:Distribution</code> o con <code>IOContract</code> si afectan a la invocación.
Reproducibilidad técnica	Dataset de entrenamiento y evaluación mediante <code>MLDCAT-AP</code> , <code>DCAT</code> y <code>PROV-O</code> ; artefactos de reproducibilidad mediante <code>it6:Flow</code> , evidencia, checksum y contexto compartido de evaluación.	Hiperparámetros, dependencias software, versiones de framework, GPU y memoria pertenecen a <code>daimo-reproducibility</code> o a reutilización más detallada de <code>ML-Schema/MEX/MLDCAT-AP</code> , no a nuevas clases núcleo.
Versionado y linaje de modelo	DAIMO puede representar derivación y revisión mediante <code>PROV-O</code> cuando el grafo lo declara, y puede auditar la cadena ejecución-artefacto-autorización.	Un perfil de linaje puede añadir reglas <code>SHACL</code> para <code>prov:wasDerivedFrom</code> , <code>prov:wasRevisionOf</code> , estado de obsolescencia y trazas de reentrenamiento sin alterar las catorce clases nativas.
Automatización MLOps	Fuera del núcleo. DAIMO define el contrato semántico que una herramienta puede poblar, pero no implementa conectores ni extracción automática.	Integradores desde Hugging Face, MLflow o registros de modelos pueden poblar grafos DAIMO y servir como validación aplicada posterior.
Monitorización y deriva	Fuera del núcleo salvo que los resultados se publiquen como evaluaciones contextualizadas.	Un perfil <code>daimo-monitoring</code> puede modelar deriva, degradación, latencia observada, huella computacional o sostenibilidad como métricas temporales o eventos de evaluación.
Seguridad y cumplimiento	El núcleo cubre política, autorización, evidencia de auditoría, checksum, firmante y timestamp. No modela vulnerabilidades, ataques o interpretación jurídica automática.	Perfiles <code>daimo-security</code> y <code>daimo-compliance</code> pueden añadir vulnerabilidades, firmas, controles <code>AI Act</code> , sesgo/equidad/explicabilidad y reglas <code>SHACL/ODRL</code> específicas de dominio.

Shapes SHACL

La capa SHACL de DAIMO contiene nueve *shapes* nodales de completitud, tres *shapes* de conformidad sobre clases reutilizadas y seis invariantes SHACL-SPARQL entre clases. El módulo `shapes/daimo-shapes.ttl` se declara a su vez como `owl:Ontology` independiente con `owl:versionIRI` bajo <https://w3id.org/pionera/daimo/shapes/>.

- **Shapes de completitud (9):**
`AIAssetOfferingShape`,
`ParticipantRoleShape`,
`ModelDeploymentShape`, `IOContractShape`,
`ExecutionAuthorizationShape`,
`DerivedArtifactShape`,
`SharedEvaluationContextShape`,
`AuditEvidenceShape` y
`CrossParticipantProvenanceRecordShape`.

- **Shapes de conformidad sobre clases reutilizadas (3):**
`OfferInDAIMOShape` exige `odrl:assigner` y `odrl:target` sobre toda `odrl:Offer`;
`MachineLearningModelInDAIMOShape` exige política, título e identificador sobre `it6:MachineLearningModel`;
`RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`.

- **Invariantes cruzadas (6):**
`DerivationAuthorizationInvariant` (INV-1),
`RunAgentAuthorizationInvariant` (INV-2),
`DeploymentServiceInvariant` (INV-3),
`AuthorizationTemporalInvariant` (INV-4),
`OfferingPolicyTargetInvariant` (INV-5) y
`OfferingAssignerInvariant` (INV-6); todas como `sh:SPARQLConstraint`.

Consultas SPARQL

Las veintitrés consultas SPARQL correspondientes a las preguntas del Apéndice están publicadas en `daimo/queries/queries.md`. El validador ejecuta cada consulta sobre el cierre OWL-RL del grafo unión ontología más ejemplo y verifica que cada una devuelve al menos una fila. Los conteos exactos aparecen en la Tabla 11. CQ-G2 devuelve dos filas —una por cada endpoint del despliegue multi-protocolo REST y gRPC—, lo que confirma que el modelado de múltiples servicios por despliegue es un patrón legítimo y directamente observable desde SPARQL.

La trazabilidad detallada se conserva en el apéndice para no sobrecargar la sección de evaluación. La tabla siguiente permite inspeccionar qué clases y propiedades ejercita cada CQ y mantiene en el PDF la evidencia mínima necesaria para reproducir la interpretación de los conteos.

La columna *Filas* de la Tabla 15 es la evidencia reproducible que respalda la afirmación de que las veintitrés preguntas de competencia se responden: cada conteo esperado de filas corresponde a las instancias declaradas en el grafo del demostrador. Como las preguntas recorren toda la cadena gobernada, desde la publicación hasta la evaluación compartida, la tabla funciona además como verificación de cobertura sobre las clases nativas y sus propiedades principales. Un lector puede así reproducir cualquier consulta individual y confirmar que devuelve resultados y que ejercita los constructos que el perfil dice conectar.

Glosario de abreviaturas

El glosario final recoge las abreviaturas que reaparecen en varias secciones del artículo, desde nombres de vocabularios hasta términos regulatorios y de razonamiento. Se mantiene como una única tabla de consulta para no repetir definiciones en el cuerpo principal y para que la terminología quede

Tabla 15. Trazabilidad CQ–clases–propiedades–resultado.

CQ	Pregunta resumida	Clases principales	Propiedades recorridas	Filas
R1	Modelos publicados por un proveedor	AIAssetOffering, MachineLearningModel	offeredBy, offersModel	3
R2	Oferta, proveedor y fecha de un modelo	AIAssetOffering, CatalogRecord	offersModel $\sqsubseteq$ foaf:primaryTopic, offeredBy, dct:issued	1
R3	Licencia y política aplicables	MachineLearningModel, odrl:Offer	hasOfferPolicy, dct:license, odrl:permission	1
R4	Contrato de entrada/salida	ModelDeployment, IOContract	hasIOContract, inputFormat, outputFormat, authMethod	2
R5	Rol concreto del proveedor	ParticipantRole, subclases de rol	offeredBy, rdf:type/rdfs:subClassOf*	2
D1	Modelos por tarea y dominio	MachineLearningModel, Task	it6:hasTask, dcat:theme	3
D2	Modelos por patrón de política	AIAssetOffering, odrl:Offer	hasOfferPolicy, odrl:permission, odrl:prohibition	2
D3	Endpoints y autenticación	ModelDeployment, DataService, IOContract	exposedAs, endpointURL, authMethod	4
D4	Umbral mínimo de métrica	Evaluation, SharedEvaluationContext	usesEvaluationContext, it6:hasEvaluationMeasure, it6:hasValue	1
E1	Endpoint desde una oferta	AIAssetOffering, ModelDeployment, IOContract	offersModel, hasDeployment, exposedAs, hasIOContract	4
E2	Ejecuciones autorizadas	ExecutionAuthorization, Run	authorizesRun, grantedTo, prov:wasAssociatedWith	1
E3	Implementación e infraestructura	Run, Flow, ComputerInfrastructure	it6:hasFlow, it6:hasAlgorithm, onInfrastructure	2
E4	Evidencia de auditoría	AuditEvidence, spdx:Checksum	evidenceOf, integrityHash, signedBy, recordedAt	1
E5	Artefactos derivados	DerivedArtifact, ExecutionAuthorization	derivedFromRun, underAuthorization	1
V1	Contexto compartido	SharedEvaluationContext, Evaluation	contextTask, contextDataset, protocol, randomSeed	1
V2	Mejor modelo bajo contexto	Evaluation, MachineLearningModel	usesEvaluationContext, it6:hasEvaluationMeasure, it6:hasValue, ORDER BY	1
V3	Ranking bajo misma métrica	Evaluation, SharedEvaluationContext, MachineLearningModel	usesEvaluationContext, it6:evaluates, it6:hasEvaluationMeasure, it6:hasValue, ORDER BY	2
V4	Benchmarks del modelo	MachineLearningModel, Benchmark	it6:hasBenchmark, dct:title opcional	1
V5	Artefactos de reproducibilidad	Flow, AuditEvidence, DerivedArtifact	records, evidenceOf, integrityHash	4
G1	Ofertas que incluyen un modelo	AIAssetOffering, MachineLearningModel	offersModel, hasOfferPolicy	1
G2	Despliegues por modelo	ModelDeployment, IOContract	deploysModel, onInfrastructure, hasIOContract	2
G3	Acuerdo que autorizó una ejecución	ExecutionAuthorization, Run, odrl:Agreement	authorizesRun, grantedTo, expiresAt	1
G4	Bundle completo de procedencia	CrossParticipant ProvenanceRecord, DerivedArtifact	records, spansParticipantContext, GROUP_CONCAT	1

alineada entre la versión maestra en español y el inglés técnico. Las entradas se ordenan alfabéticamente para agilizar las búsquedas individuales durante la revisión. Nada en el glosario introduce nuevas afirmaciones; sólo fija las expansiones usadas en el resto del texto.

Como la Tabla 16 mezcla nombres de vocabularios, herramientas y referencias regulatorias, también registra el alcance exacto en que se usa cada acrónimo en este artículo. Cuando una abreviatura pudiera ser ambigua fuera del contexto de espacios de datos, la expansión de la tabla es la lectura pretendida a lo largo del texto. Se espera que el glosario permanezca estable entre versiones del perfil, de modo que revisiones posteriores puedan ampliarlo sin redefinir las entradas existentes.

Convenciones de URI

DAIMO utiliza el namespace canónico <https://w3id.org/pionera/daimo>, con identificadores de términos en la forma <https://w3id.org/pionera/daimo#<Término>>

e IRIs versionadas bajo <https://w3id.org/pionera/daimo/<versión>>, enlazadas entre sí mediante owl:priorVersion. Los módulos satélite son ontologías independientes con IRIs versionadas propias: la capa SHACL bajo <https://w3id.org/pionera/daimo/shapes/> y el módulo de alineamientos bajo <https://w3id.org/pionera/daimo/align> (este último declarando owl:imports sobre la ontología base). El grafo de ejemplo utiliza el namespace local <https://example.org/daimo-scenario/>, claramente separado del espacio canónico. La negociación de contenido está diseñada para resolver el mismo identificador a HTML, Turtle, RDF/XML, JSON-LD o N-Triples en función del Accept header. Hasta que el redireccionamiento se registre formalmente, el repositorio fuente y la versión archivada proporcionan los puntos de acceso de referencia para revisión y replicación.

Tabla 16. Abreviaturas utilizadas en el artículo.

Abreviatura	Significado
AI Act	Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence.
CQ	Competency Question.
DAIMO	Dataspace AI Model Ontology.
DCAT	Data Catalog Vocabulary.
DCAT-AP	DCAT Application Profile.
DL	Description Logic.
DOI	Digital Object Identifier.
DPV	Data Privacy Vocabulary.
DSP	Dataspace Protocol.
EDC	Eclipse Dataspace Components.
FAIR	Findable, Accessible, Interoperable, Reusable.
LOT	Linked Open Terms.
MLDCAT-AP	Machine Learning Data Catalogue Application Profile.
ODRL	Open Digital Rights Language.
OOPS!	Ontology Pitfall Scanner.
OWL	Web Ontology Language.
PROV-O	PROV Ontology.
RDF	Resource Description Framework.
RDFS	RDF Schema.
SHACL	Shapes Constraint Language.
SKOS	Simple Knowledge Organization System.
SPARQL	SPARQL Protocol and RDF Query Language.
SWJ	Semantic Web Journal.
TTL	Turtle.
WIDOCO	Wizard for Documenting Ontologies.

La elección de IRIs con # se limita a términos de vocabulario, no a instancias operativas de alto volumen. En un espacio de datos productivo, modelos concretos, datasets, ejecuciones y artefactos derivados deben identificarse bajo namespaces de los participantes o del catálogo federado; DAIMO sólo proporciona los predicados y clases con los que esos recursos se describen. Los términos nativos usan inglés ASCII: clases en *UpperCamelCase* singular (*AIAssetOffering*, *ModelDeployment*), propiedades en *lowerCamelCase* verbal o relacional (*offersModel*, *authorizesRun*, *usesEvaluationContext*) y módulos/versiones mediante segmentos *slash*. Las traducciones, sinónimos y explicaciones se expresan con `rdfs:label`, `rdfs:comment` y SKOS, no mediante variantes de URI.